

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Curso Académico 2026-2027

Trabajo de Fin de Grado

Diseño de un pipeline experimental reproducible para la clasificación de fake news mediante modelos de aprendizaje automático

Autor: Álvaro Osuna Flores

Tutora: Liliana Patricia Santacruz Valencia

Tabla de contenido

Resumen....iv
1. Introducción.... 1
1.1 Motivación....2
1.2 Objetivos....3
1.3 Estructura de la memoria....4
2. Contexto....4
2.1 Desinformación y concepto operativo de fake news....4
2.2 Clasificación automática de textos y procesamiento del lenguaje natural....5
2.3 Conjuntos de datos, corpus, anotación y transformación de etiquetas....6
2.4 Representación TF-IDF y clasificadores lineales....7
2.5 Evaluación, reproducibilidad y amenazas a la validez....8
2.6 Trabajos relacionados....9
2.7 Telegram como dominio de adquisición futura....11
2.8 Posicionamiento y alcance del TFG....12
3. Metodología, requisitos y arquitectura funcional....13
3.1 Metodología de trabajo....13
3.2 Requisitos funcionales....14
3.3 Requisitos no funcionales....15
3.4 Arquitectura funcional....16
3.5 Herramientas y tecnologías....17
3.6 Modelo de datos y gobierno del corpus....18
3.7 Reproducibilidad y control de cambios....21
3.8 Decisiones de diseño y justificación técnica....22
4. Desarrollo e implementación....23
4.1 Construcción del corpus derivado....23
4.2 Validación del corpus....28
4.3 Preprocesamiento textual....30
4.4 Módulo complementario de adquisición desde Telegram....32
4.5 Configuración de los experimentos....33
4.6 Artefactos de trazabilidad....35
4.7 Protocolo de reproducción....39
4.8 Implementación por módulos....40

4.9 Pruebas y control de calidad....42
5. Evaluación experimental y análisis de resultados....44
5.1 Comparativa de modelos candidatos....44
5.2 Métricas del modelo seleccionado....46
5.3 Matriz de confusión....47
5.4 Análisis cualitativo de errores....49
5.5 Interpretabilidad del modelo lineal....50
5.6 Evaluación de transferencia entre fuentes....51
5.7 Validación automatizada....52
5.8 Amenazas a la validez....52
5.9 Interpretación y límites de validez....54
5.10 Criterios de aceptación del protocolo....54
6. Conclusiones y líneas futuras....55
Bibliografía....58
Glosario....61
Apéndice A. Evidencia histórica del baseline de una partición....62

Resumen

La difusión de desinformación a través de medios digitales ha impulsado el desarrollo de sistemas automáticos para clasificar contenidos potencialmente falsos. Sin embargo, un rendimiento elevado puede deberse a diferencias de fuente, formato o estilo editorial presentes en los datos y no únicamente a señales generalizables de desinformación. Este problema dificulta la interpretación de los resultados y puede generar una percepción errónea sobre la capacidad real de los clasificadores.

Ante esta situación, el presente Trabajo Fin de Grado propone una metodología experimental reproducible para la clasificación textual de *fake news*, con especial atención a la trazabilidad de los datos, las transformaciones aplicadas y los resultados obtenidos.

Para ello, a partir de 50.503 registros de archive2 y LIAR se construyó un corpus binario derivado de 44.408 representantes tras auditar duplicados y excluir grupos de etiquetas contradictorias. El flujo incorpora validaciones de esquema, integridad, etiquetas y valores nulos; aplica un preprocesamiento determinista y representa los textos mediante TF-IDF. Se compararon regresión logística y Linear SVC mediante cinco particiones agrupadas comunes, con ajuste del vectorizador exclusivamente en los datos de entrenamiento. La evaluación incluye métricas, matrices de confusión, análisis de errores, términos influyentes y pruebas de transferencia entre fuentes. Se conservaron configuración, particiones, predicciones, modelos y manifiestos para auditar cada ejecución.

La regresión logística obtuvo el mayor macro F1 medio de la comparación interna en cinco particiones agrupadas, 0,9392 ± 0,0013, frente a 0,9388 ± 0,0020 de Linear SVC. El macro F1 de 0,9424 de Linear SVC corresponde al *holdout* anterior y se conserva como referencia histórica bajo un protocolo distinto. No obstante, el análisis muestra que los modelos pueden aprovechar marcas de procedencia y patrones editoriales, por lo que estas métricas no demuestran generalización a otras fuentes o dominios. El módulo complementario de adquisición de mensajes de Telegram permaneció separado del experimento, dado que sus datos no estaban etiquetados.

En conclusión, el trabajo proporciona una línea base eficiente, interpretable y reproducible para estudiar la clasificación textual de *fake news*. Su principal aportación reside en combinar el rendimiento predictivo con la trazabilidad experimental y con una delimitación explícita de las condiciones bajo las que pueden interpretarse los resultados.

Palabras clave: fake news; clasificación textual; aprendizaje automático; TF-IDF; reproducibilidad; procesamiento del lenguaje natural

1. Introducción

La expansión de las redes sociales y de las plataformas digitales ha transformado la forma en que se produce, comparte y consume información. Aunque estos medios facilitan el acceso inmediato a una gran cantidad de contenidos, también favorecen la rápida difusión de información falsa, manipulada o engañosa. Este fenómeno, habitualmente asociado al concepto de *fake news*, puede afectar a la calidad de la información disponible y dificultar que los usuarios distingan entre contenidos fiables y desinformación (Lazer et al., 2018). Además, diversos estudios han mostrado que las noticias falsas pueden propagarse en las redes sociales con mayor rapidez y alcanzar a más usuarios que las informaciones verdaderas (Vosoughi et al., 2018).

Ante este problema, la detección automática de *fake news* se ha convertido en un ámbito relevante de investigación. El procesamiento del lenguaje natural y las técnicas de aprendizaje automático permiten analizar grandes cantidades de contenido textual e identificar patrones que pueden ayudar a diferenciar entre distintos tipos de información. Sin embargo, obtener buenos resultados de clasificación no implica necesariamente que un modelo sea capaz de reconocer la desinformación de forma generalizable.

Uno de los principales retos se encuentra en los propios datos utilizados para entrenar y evaluar estos sistemas. Aspectos como la procedencia de las noticias, el estilo de redacción, la longitud de los textos, la composición de los conjuntos de datos o los criterios empleados para asignar las etiquetas pueden introducir patrones que el modelo utilice para realizar la clasificación sin que estos estén directamente relacionados con la veracidad del contenido. Por este motivo, la evaluación de un sistema de detección de *fake news* no debe limitarse únicamente a analizar sus métricas de rendimiento, sino que también debe considerar cómo se han construido y procesado los datos utilizados en el experimento (Shu et al., 2017; Thorne & Vlachos, 2018).

En este contexto, resulta especialmente importante disponer de un proceso experimental claro, trazable y reproducible. La documentación de las fuentes de datos, los criterios de selección y etiquetado, las transformaciones realizadas sobre los textos, la división del corpus y la configuración de los modelos permite interpretar adecuadamente los resultados obtenidos y facilita la repetición y verificación de los experimentos.

Partiendo de esta necesidad, este Trabajo Fin de Grado aborda la detección automática de *fake news* mediante el diseño y desarrollo de un proceso experimental reproducible para la construcción, preparación y análisis de un corpus textual. El trabajo no se centra únicamente en

obtener un modelo con buenas métricas de clasificación, sino también en estudiar de forma controlada cómo las características y transformaciones de los datos pueden influir en los resultados. De este modo, se pretende proporcionar una base experimental que permita evaluar los modelos de forma más rigurosa y comprender mejor las limitaciones asociadas al proceso de detección automática de desinformación.

1.1 Motivación

La detección automática de *fake news* plantea un problema especialmente relevante desde el punto de vista del procesamiento del lenguaje natural y el aprendizaje automático. La gran cantidad de información que circula a través de medios digitales hace inviable realizar una revisión manual de todos los contenidos, por lo que resulta de interés estudiar métodos capaces de apoyar su análisis y clasificación de forma automatizada.

Sin embargo, uno de los principales desafíos de estos sistemas consiste en determinar hasta qué punto los resultados obtenidos reflejan realmente la capacidad del modelo para identificar patrones relacionados con la desinformación. El rendimiento puede verse condicionado por factores asociados a los datos de entrenamiento, como su procedencia, el estilo de redacción, el formato de los documentos o los criterios utilizados para establecer las etiquetas. Como consecuencia, un modelo puede alcanzar resultados elevados aprovechando características particulares del conjunto de datos sin que ello implique necesariamente una buena capacidad de generalización.

Esta problemática motiva el desarrollo de un entorno experimental en el que sea posible controlar y documentar las diferentes etapas del proceso de clasificación. En este TFG se plantea, por tanto, la construcción de un pipeline reproducible que permita integrar diferentes conjuntos de datos públicos, preparar y representar los textos, entrenar modelos de clasificación bajo unas condiciones comunes y analizar sus resultados de manera sistemática. Más que buscar exclusivamente el modelo con mejores métricas, se pretende disponer de un proceso que permita comprender e interpretar los resultados obtenidos y estudiar la influencia que pueden ejercer las características del corpus sobre ellos.

Adicionalmente, el interés por estudiar este problema en entornos de comunicación actuales ha llevado a incorporar un módulo para la adquisición de mensajes procedentes de canales públicos de Telegram. Aunque estos datos no forman parte del entrenamiento ni de la evaluación realizada en este trabajo, el módulo establece una primera base técnica para futuras extensiones

en las que pueda estudiarse el comportamiento de los modelos sobre información procedente de esta plataforma.

De este modo, la motivación del proyecto combina el interés social asociado a la detección de desinformación con un problema técnico relevante: desarrollar procesos de clasificación que, además de proporcionar resultados cuantitativos, sean reproducibles, trazables y permitan analizar de forma crítica las posibles limitaciones derivadas de los datos utilizados.

1.2 Objetivos

Los objetivos definidos para el presente proyecto se describen a continuación.

Objetivo general

Diseñar, implementar y evaluar un proceso experimental reproducible para la clasificación automática de *fake news* a partir de contenido textual, mediante la construcción y validación de un corpus unificado y el entrenamiento y comparación de modelos supervisados de clasificación.

Objetivos específicos

- Construir y validar un corpus binario unificado a partir de conjuntos de datos públicos, estableciendo criterios explícitos de selección, inclusión, exclusión y transformación de etiquetas.
- Diseñar e implementar un pipeline de procesamiento textual reproducible, que permita realizar de forma determinista las tareas de limpieza, normalización y representación de los textos.
- Implementar mecanismos automáticos de control de calidad del corpus, verificando aspectos como la estructura de los datos, la integridad de los registros, las etiquetas, los valores nulos y la presencia de duplicados.
- Entrenar y comparar diferentes modelos supervisados de clasificación, utilizando representaciones textuales basadas en TF-IDF y una configuración experimental común.
- Evaluar el comportamiento de los modelos mediante métricas adecuadas para problemas de clasificación binaria, complementando el análisis cuantitativo con el estudio de los errores de clasificación y de los términos con mayor influencia en las predicciones.
- Garantizar la trazabilidad y reproducibilidad de los experimentos, conservando la configuración utilizada, las métricas obtenidas, las predicciones, las matrices de confusión y los artefactos necesarios para reproducir y revisar cada ejecución.

- Implementar, como extensión del sistema experimental, un módulo para la adquisición bajo demanda de mensajes procedentes de canales públicos de Telegram, que permita su extracción, normalización y deduplicación como base para futuros estudios sobre este dominio.

1.3 Estructura de la memoria

La estructura de la memoria es como sigue:

La memoria se organiza en seis capítulos. El capítulo 2, Contexto, presenta los fundamentos sobre desinformación, clasificación textual, conjuntos de datos, modelos lineales, reproducibilidad y trabajos relacionados. El capítulo 3, Metodología, requisitos y arquitectura funcional, describe el protocolo agrupado, los requisitos y la arquitectura. El capítulo 4, Desarrollo e implementación, detalla la construcción y validación del corpus derivado, el preprocesamiento, el entrenamiento, los artefactos y el módulo complementario de Telegram. El capítulo 5, Evaluación experimental y análisis de resultados, examina la comparación de modelos, las transferencias entre fuentes, los errores, la interpretabilidad y las limitaciones; identifica por separado el *baseline* histórico. El capítulo 6, Conclusiones y líneas futuras, sintetiza las contribuciones, el alcance y las posibles extensiones.

El apéndice A conserva la configuración, las métricas por clase, la matriz de confusión y los términos del *baseline* histórico como referencia documental separada del experimento agrupado principal.

2. Contexto

Este capítulo reúne los conceptos que sustentan la metodología experimental del TFG. Se presentan primero la definición operativa del problema, la clasificación automática de textos, los corpus empleados y los modelos lineales; después se abordan la reproducibilidad, los trabajos relacionados, Telegram como dominio posible y los límites concretos del estudio.

2.1 Desinformación y concepto operativo de *fake news*

La desinformación es un fenómeno comunicativo y técnico en el que intervienen la intención, el contenido, el contexto de difusión y la recepción social. Lazer et al. (2018) subrayan que su estudio exige datos compartibles, métodos verificables y análisis empírico. Desde una perspectiva computacional, Shu et al. (2017) distinguen enfoques basados en el contenido, el

contexto social y la propagación. Estas dimensiones muestran que la veracidad no puede deducirse de una propiedad textual aislada y que cualquier clasificador debe interpretarse dentro de las condiciones en las que se construyó su corpus.

En este trabajo, *fake news* se utiliza como una categoría operativa de clasificación binaria derivada de las etiquetas proporcionadas por los conjuntos de datos de origen. El modelo aprende a separar registros etiquetados como `fake_news` de registros etiquetados como `no_fake`; no comprueba de forma independiente cada afirmación ni recupera evidencias externas. Esta delimitación distingue la clasificación supervisada del *fact-checking* automático, que comprende la identificación de afirmaciones verificables, la recuperación de evidencias y la determinación de si esas evidencias apoyan o contradicen una afirmación (Thorne & Vlachos, 2018).

2.2 Clasificación automática de textos y procesamiento del lenguaje natural

La clasificación automática de textos asigna una etiqueta a un documento a partir de una representación numérica. El flujo habitual limpia y normaliza el texto, lo transforma en características y ajusta un modelo con ejemplos anotados. Los fundamentos de categorización textual describen distintas representaciones y algoritmos, y destacan que el protocolo de preprocesamiento forma parte de la definición experimental, no de una etapa neutra (Sebastiani, 2002). En este trabajo, las herramientas de aprendizaje automático se emplean mediante implementaciones documentadas y versionadas (Pedregosa et al., 2011).

Las decisiones de preprocesamiento forman parte del modelo experimental porque alteran la información disponible para el clasificador. La decodificación del lenguaje de marcado de hipertexto (HTML), el tratamiento de los localizadores uniformes de recursos (URL), las menciones y los hashtags, la normalización de mayúsculas y la selección de *stopwords* se aplican siempre con las mismas reglas. También es necesario registrar cuándo un texto queda vacío, resulta demasiado corto o presenta un idioma incierto. Un flujo determinista evita que dos ejecuciones comparables generen representaciones distintas por cambios implícitos en la limpieza.

Los métodos basados en contenido pueden utilizar rasgos léxicos, sintácticos o estilísticos. Son apropiados cuando solo se dispone del texto, pero no representan por sí solos la difusión, la credibilidad de la fuente o la respuesta de los usuarios. Los enfoques sociales y de propagación incorporan esas señales adicionales, aunque requieren datos que pueden ser incompletos,

ruidosos o difíciles de obtener de manera homogénea (Shu et al., 2017). El presente TFG se sitúa deliberadamente en la familia basada en contenido textual. Los análisis de estilo advierten que las marcas editoriales pueden ser predictivas dentro de un corpus y no necesariamente transferibles entre dominios (Potthast et al., 2018).

Los modelos de lenguaje preentrenados, como *Bidirectional Encoder Representations from Transformers* (BERT), generan representaciones contextuales capaces de distinguir usos diferentes de una palabra y han mejorado numerosas tareas de procesamiento del lenguaje natural (Devlin et al., 2019). Sin embargo, su mayor complejidad computacional y experimental dificulta aislar el efecto de los datos, el ajuste y la arquitectura. Por ello, antes de incorporar modelos profundos resulta útil establecer una línea base lineal que sea ligera, explicable y reproducible.

La desinformación también puede presentarse mediante imágenes, vídeos, audios, capturas o combinaciones de modalidades. Un sistema exclusivamente textual no analiza esos elementos y tampoco reconstruye la propagación de un mensaje. Estas líneas son pertinentes para situar el problema, pero quedan fuera del alcance evaluado: el experimento estudia clasificación textual supervisada y no pretende cubrir todas las formas posibles de desinformación.

2.3 Conjuntos de datos, corpus, anotación y transformación de etiquetas

El corpus evaluado se deriva de dos conjuntos de datos públicos etiquetados. LIAR contiene declaraciones políticas breves de PolitiFact distribuidas en seis niveles (Wang, 2017); archive2 redistribuye los ficheros `Fake.csv` y `True.csv` atribuidos al conjunto ISOT (Ahmed et al., 2018; emineyetm, 2022). La combinación reúne longitudes, formatos, estilos y criterios de anotación diferentes, por lo que cada fuente se conserva como metadato.

Para formular un problema binario, en archive2 las etiquetas Fake se asignan a `fake_news` y `True` a `no_fake`. En LIAR se asignan `false` y `pants-fire` a `fake_news`, y `true` a `no_fake`; `half-true`, `mostly-true` y `barely-true` se excluyen por ser categorías intermedias. En la nueva versión derivada, `no_fake` identifica exclusivamente la clase negativa según las etiquetas recibidas y no significa que cada registro haya sido verificado de manera independiente. La transformación reduce la granularidad original de LIAR y no establece una equivalencia semántica universal.

Un conjunto de datos es una fuente concreta con su propio esquema y proceso de anotación, mientras que el corpus unificado resulta de integrar, filtrar, deduplicar y normalizar las fuentes.

La procedencia permite estudiar la composición de las clases y los errores. Los estudios comparativos muestran que las diferencias entre conjuntos y criterios pueden influir en el rendimiento, y que una representación estilística puede separar fuentes sin representar por ello veracidad (Gravanis et al., 2019; Potthast et al., 2018).

La integración exige un esquema común, reglas de etiqueta trazables y un inventario de inclusión, duplicación y exclusión. Las declaraciones de datos y las hojas descriptivas de conjuntos de datos proponen registrar decisiones de composición, procedencia y limitaciones para hacer visibles los supuestos de una evaluación (Bender & Friedman, 2018; Gebru et al., 2021). Los controles estructurales de este proyecto detectan incoherencias de forma, pero no determinan la veracidad de una noticia ni corrigen posibles errores de anotación.

2.4 Representación TF-IDF y clasificadores lineales

La representación TF-IDF pondera cada término atendiendo a su frecuencia en el documento y a su rareza en el conjunto de documentos. De este modo, una palabra repetida dentro de una noticia puede recibir peso, mientras que su presencia en muchos documentos reduce su capacidad discriminante. Esta familia de ponderación se desarrolló en recuperación de información y sigue siendo una línea base interpretable para clasificación textual (Salton & Buckley, 1988; Sebastiani, 2002). El vector resultante no codifica comprensión factual del contenido.

El uso de unigramas y bigramas amplía la información disponible para el clasificador. Un unigrama captura palabras aisladas, mientras que un bigrama conserva pares consecutivos. Esa decisión resulta relevante porque varias señales de procedencia o estilo aparecen como expresiones compuestas, por ejemplo, *washington reuters*, *read more*, *featured image* u *on wednesday*. Si el vectorizador solo utilizara palabras aisladas, parte de esas señales quedaría fragmentada. Al incorporar bigramas, el modelo recibe patrones más cercanos a la forma real de los textos.

La regresión logística y el clasificador lineal de vectores de soporte (Linear SVC) se utilizan en espacios dispersos de alta dimensión. Los clasificadores de margen se fundamentan en máquinas de vectores de soporte, mientras que su aplicación a categorización textual es adecuada cuando existen numerosas características léxicas (Cortes & Vapnik, 1995; Joachims, 1998). La comparación con una regresión logística proporciona un contraste lineal bajo la misma representación, sin convertir los coeficientes en causas de la etiqueta.

La Regresión Logística produce probabilidades que pueden compararse con un umbral configurable. Linear SVC, en cambio, optimiza el margen de separación entre clases y genera una función de decisión, cuya puntuación no equivale a una probabilidad calibrada. Los umbrales documentados en el proyecto son decisiones de diseño conservadoras para la partición y la configuración utilizadas; no son valores aprendidos ni garantías aplicables a otros corpus.

La línea base lineal no se plantea como una solución definitiva. Su función es establecer una referencia reproducible, ligera y explicable. Si posteriormente se incorporara un *transformer*, debería compararse utilizando las mismas particiones, fuentes y reglas de evaluación. De otro modo, una mejora podría deberse a cambios en el corpus, el preprocesamiento o el protocolo y no necesariamente al modelo.

2.5 Evaluación, reproducibilidad y amenazas a la validez

La evaluación de una clasificación binaria requiere distinguir verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos. Métricas como precisión, recall y F1 responden de forma diferente a los cambios de la matriz de confusión y al reparto de clases, de modo que su interpretación debe corresponder al objetivo evaluado (Sokolova & Lapalme, 2009). Este trabajo informa macro F1 como criterio de comparación y complementa la media con dispersión entre particiones y matrices de confusión.

Una evaluación con grupos requiere que los registros relacionados permanezcan en una única partición. Las estrategias de validación cruzada deben respetar dependencias temporales, espaciales o jerárquicas cuando estas están presentes (Roberts et al., 2017). Además, el uso de información duplicada entre entrenamiento y prueba puede provocar fuga de datos y estimaciones optimistas (Kapoor & Narayanan, 2023; Kaufman et al., 2012). Por esta razón, el protocolo actual detecta duplicados exactos y casi duplicados antes de crear cinco particiones estratificadas por grupo.

La trazabilidad relaciona configuración, huellas de los datos y del código, identificador de ejecución, particiones, predicciones, modelos y métricas. Un manifiesto verificable facilita auditar si una cifra pertenece al corpus y al protocolo que el texto declara. Esta transparencia no elimina sesgos ni garantiza replicación independiente, pero permite inspeccionar las decisiones y detectar cambios de entrada o implementación (Bender & Friedman, 2018; Gebru et al., 2021; Kapoor & Narayanan, 2023).

El análisis de términos puede revelar marcas de agencia, cabeceras, fórmulas periodísticas o expresiones propias de una fuente. Si esas marcas se distribuyen de forma desigual entre clases, el clasificador puede utilizarlas como atajos. Este sesgo de fuente limita la validez externa incluso cuando el rendimiento interno es elevado. Por ello, la composición del corpus, la transformación de etiquetas y la partición deben documentarse junto a las métricas, no como información secundaria.

La exposición de estas limitaciones también forma parte de la ética de datos. El sistema no automatiza decisiones sobre personas, canales o comunidades y no se ha validado para un despliegue real. Presentarlo como detector general ocultaría la simplificación de LIAR, las diferencias editoriales entre fuentes y la ausencia de evaluación en otros dominios. La reproducibilidad permite auditar el experimento, pero no sustituye una validación externa.

2.6 Trabajos relacionados

La detección automática de *fake news* se ha estudiado desde el procesamiento del lenguaje natural, la minería de datos y el análisis de redes. Las revisiones organizan el campo por tarea, modalidad, conjunto de datos y método, y destacan que los resultados dependen de esos elementos (Oshikawa et al., 2020; Shu et al., 2017; Zhou & Zafarani, 2020). Por esa razón, la Tabla 1 contrasta dimensiones metodológicas, no puntuaciones aisladas.

También cambia la propia formulación de la tarea. Algunos trabajos predicen dos clases, otros conservan escalas de veracidad y otros analizan credibilidad, propagación o apoyo de evidencias. Del mismo modo, *accuracy*, macro F1 y otras métricas responden de forma distinta al desequilibrio y a la definición de las clases. Una comparación académica útil debe describir estas condiciones antes de contraponer cifras; trasladar un resultado sin su corpus y protocolo produciría una equivalencia metodológica inexistente.

Los enfoques textuales clásicos convierten las noticias en vectores y aplican algoritmos supervisados. Ozbay y Alatas (2020) comparan clasificadores sobre representaciones de ponderación de términos. Pérez-Rosas et al. (2018) describen la recogida y anotación de corpus en varios dominios y muestran la importancia de validar las etiquetas. Estos antecedentes motivan que la memoria detalle tanto el método como las fuentes y exclusiones.

Los enfoques profundos pueden integrar información adicional. Wang (2017) acompaña la publicación de LIAR con experimentos lingüísticos y un modelo híbrido que combina texto y metadatos. Ruchansky et al. (2017) proponen CSI, que utiliza el contenido, la respuesta

temporal de los usuarios y características de la fuente. Estos diseños responden a objetivos diferentes de una línea base exclusivamente textual y requieren datos que el corpus de este TFG no contiene.

Otras líneas incorporan señales multimodales o formulan la verificación de afirmaciones mediante recuperación de evidencias. Los trabajos sobre diferencias de dominio advierten que el rendimiento puede cambiar cuando cambian el tema y las características del conjunto de evaluación (Silva et al., 2021), mientras que el *fact-checking* persigue una tarea distinta de asignar una categoría aprendida a un texto (Thorne & Vlachos, 2018). Este TFG no implementa esas funciones.

La Tabla 1 compara cuatro antecedentes y el presente TFG en cuanto a corpus, representación, evaluación y límites. La síntesis no ordena las obras por una métrica compartida: explica qué decisiones metodológicas de esta memoria responden a sus diferencias.

Tabla 1

Comparación de trabajos relacionados por dimensiones metodológicas

Trabajo	Corpus y representación	Modelo y evaluación	Reproducibilidad y limitaciones
Shu et al. (2017)	Revisión de conjuntos de datos, contenido y contexto social.	Taxonomía de métodos, métricas y recursos; no presenta un experimento único.	Marco conceptual; destaca datos sociales incompletos, no estructurados y ruidosos.
Wang (2017)	LIAR: declaraciones breves de PolitiFact, seis etiquetas, texto y metadatos.	Baselines lingüísticos y modelo híbrido de red neuronal convolucional (CNN); evaluación multiclase.	Conjunto de datos público; la binarización del TFG reduce la granularidad original.
Ruchansky et al. (2017)	Noticias e interacciones de Twitter y Weibo; texto, respuesta temporal y fuente.	Modelo profundo CSI evaluado frente a baselines mediante accuracy.	No es comparable directamente con un pipeline solo textual porque requiere contexto social.
Ozbay y Alatas (2020)	Tres conjuntos de datos públicos representados mediante ponderación TF y matriz documento-término.	Veintitrés clasificadores supervisados comparados mediante cuatro métricas.	Comparación amplia; los resultados dependen de corpus y protocolo distintos.
Presente TFG	archive2 + LIAR; corpus binario derivado, TF-IDF de unigramas y bigramas.	Regresión logística y Linear SVC; cinco particiones agrupadas y transferencias en ambas direcciones.	Comparación interna y alcance acotado a estas fuentes; la búsqueda MinHash es aproximada; Telegram queda excluido.

Nota. Los resultados no son directamente comparables debido a las diferencias de corpus, representación, modelos y protocolos de evaluación.

2.7 Telegram como dominio de adquisición futura

Telegram permite publicar mensajes en canales públicos, reenviar contenidos, incorporar material multimedia y acceder a determinadas publicaciones mediante una interfaz de programación de aplicaciones (API). Estas propiedades lo convierten en un posible dominio de adquisición para estudiar desinformación. Sin embargo, sus mensajes pueden ser breves, fragmentados, contextuales o acompañados de enlaces, emojis e imágenes, y no comparten necesariamente la distribución de los artículos de archive2 ni de las declaraciones de LIAR.

El módulo implementado realiza adquisiciones bajo demanda en los canales públicos configurados y genera ficheros JSON que pueden normalizarse y deduplicarse. Su salida permanece sin etiquetar y no se incorporó al corpus unificado, al entrenamiento ni a la evaluación descritos en la memoria. Por tanto, el módulo es una capacidad técnica independiente; no constituye monitorización continua, detección en tiempo real ni evidencia de que el modelo se transfiera al dominio de Telegram.

Una evaluación futura en este dominio exigiría seleccionar mensajes, establecer instrucciones de anotación, resolver desacuerdos entre anotadores y documentar controles de calidad. También deberían definirse medidas de minimización, anonimización y revisión ética cuando proceda. Hasta completar ese proceso, los mensajes adquiridos solo pueden apoyar la preparación de un corpus futuro y el estudio posterior de transferencia de dominio.

2.8 Posicionamiento y alcance del TFG

El proyecto se posiciona como una base experimental reproducible para estudiar clasificación textual de *fake news*. El valor principal reside en integrar fuentes etiquetadas mediante reglas explícitas, validar el corpus, aplicar un preprocesamiento determinista, comparar dos modelos lineales y conservar los artefactos de cada ejecución. La interpretabilidad de la línea base forma parte del diseño porque permite examinar si el modelo utiliza contenido relevante o marcas superficiales.

Esta delimitación no reduce la utilidad del trabajo, sino que identifica qué afirmaciones pueden sostenerse con la evidencia disponible. Una línea base controlada permite localizar problemas de limpieza, evaluar la influencia de la fuente y establecer un punto de comparación para ampliaciones posteriores. Su valor reside en que cada decisión relevante queda documentada y puede revisarse. La reproducibilidad convierte el resultado en una referencia acumulable, mientras que la declaración expresa de límites evita confundir rendimiento interno con capacidad general de detectar desinformación.

El alcance evaluado se limita a archive2 y LIAR: cinco particiones estratificadas por grupos de duplicados y dos pruebas de transferencia entre esas fuentes. Las métricas describen ese marco experimental y no demuestran generalización a otros periodos, idiomas o plataformas. En particular, no se ha validado la transferencia a Telegram ni se ha construido un detector universal de noticias falsas.

Tampoco se ha implementado un sistema de *fact-checking*, un clasificador multimodal o una arquitectura de producción. Una futura ampliación exigiría corpus anotados de destino, particiones por periodo y una evaluación independiente; modelos *transformer* o recuperación de evidencias serían líneas posteriores, no prestaciones demostradas por el experimento actual.

Esta delimitación conduce directamente a los requisitos del sistema: integridad del corpus, determinismo, desacoplamiento de la adquisición, persistencia de artefactos y evaluación auditable. El capítulo 3 traduce estos principios en la metodología de trabajo, los requisitos funcionales y no funcionales y la arquitectura que organiza los componentes del pipeline.

3. Metodología, requisitos y arquitectura funcional

3.1 Metodología de trabajo

El método se organiza en preparación de datos, validación, representación, ajuste y evaluación. Primero se conserva la fuente y se genera una auditoría fila a fila; después se agrupan duplicados antes de crear particiones; cada modelo se ajusta dentro de cada *fold*; y las predicciones fuera de entrenamiento permiten calcular métricas agregadas. Una ruta separada evalúa la transferencia entre archive2 y LIAR, sin conectar Telegram con la evaluación.

La preparación, la validación cruzada y la transferencia son etapas separadas con estado persistido. Una ejecución interrumpida solo reutiliza una etapa completa cuando las huellas de entrada, configuración, código y versiones coinciden; los resultados históricos permanecen en sus propios directorios. Esta estructura permite auditar qué cambios del corpus o del protocolo exigen un nuevo identificador de ejecución.

El flujo se organiza como una cadena de evidencias. La construcción histórica aporta el corpus de entrada y su metadata; la preparación derivada registra cada inclusión, duplicado exacto y exclusión por conflicto; el agrupamiento define las unidades de partición; y cada *fold* ajusta representación y clasificador solo con sus datos de entrenamiento. La evaluación interpreta predicciones fuera de entrenamiento y transferencias entre fuentes, nunca una partición *holdout* del protocolo anterior como si fuera actual.

Cada transición mantiene un contrato explícito. La preparación exige columnas y `source_id` únicos, asigna un estado a cada fila original y produce grupos y representantes. La ejecución comprueba que ningún grupo se reparta entre entrenamiento y prueba, persiste las asignaciones y registra modelo, umbral, iteraciones, avisos, predicciones y métricas. El contrato permite

reconstruir el recorrido hasta cada cifra, sin equiparar control estructural con veracidad de las etiquetas.

La secuencia delimita las responsabilidades de cada componente y las condiciones para repetir el experimento. La adquisición de Telegram produce mensajes sin etiqueta y permanece desacoplada; la evaluación principal utiliza únicamente el corpus derivado de archive2 y LIAR.

3.2 Requisitos funcionales

La Tabla 2 recoge los requisitos funcionales en el orden del flujo experimental. La columna de trazabilidad vincula cada requisito con las secciones, evidencias o pruebas existentes y, cuando procede, con el capítulo de evaluación.

Tabla 2

Requisitos funcionales del sistema experimental

Código	Descripción	Trazabilidad
RF-1	Construcción del corpus. Integrar archive2 y LIAR, conservar procedencia y etiquetas originales, auditar inclusiones y exclusiones, y agrupar duplicados antes de particionar.	Diseño/desarrollo: 3.6 y 4.1. Evidencia: corpus, metadata y tests de construcción. Evaluación: 5.8 y 5.10.
RF-2	Preprocesamiento. Limpiar, normalizar y preparar los textos de forma determinista.	Diseño/desarrollo: 3.1 y 4.3. Evidencia: preprocessing.py y test_preprocessing.py. Evaluación: 5.7.
RF-3	Validación. Comprobar esquema, tipos, integridad, etiquetas válidas, valores nulos, duplicados y condiciones previas al entrenamiento.	Diseño/desarrollo: 3.6 y 4.2. Evidencia: validate_dataset.py y tests de validación. Evaluación: 5.7 y 5.10.
RF-4	Entrenamiento. Ajustar TF-IDF y los dos clasificadores lineales exclusivamente en el entrenamiento de cada una de cinco particiones agrupadas compartidas.	Diseño/desarrollo: 3.7 y 4.5. Evidencia: configuración, script y tests de entrenamiento. Evaluación: 5.1-5.2.

Código	Descripción	Trazabilidad
RF-5	Comparación y evaluación. Informar métricas por partición y por fuente, matrices, errores y transferencia en ambas direcciones; conservar el análisis de términos históricos como tal.	Desarrollo/evaluación: 4.6 y 5.1-5.5. Evidencia: métricas, matriz, ejemplos, términos y tests de análisis.
RF-6	Persistencia. Guardar configuración, métricas, predicciones, modelos y demás artefactos experimentales relevantes.	Diseño/desarrollo: 3.7 y 4.6. Evidencia: manifiesto, predicciones y artefactos. Evaluación: 5.7 y 5.10.
RF-7	Reproducción. Reanudar únicamente etapas completas con huellas coincidentes y permitir auditar datos, grupos, particiones, parámetros, modelos y predicciones.	Diseño/desarrollo: 3.7 y 4.7. Evidencia: configuración, manifiesto y protocolo. Evaluación: 5.7 y 5.10.
RF-8	Adquisición opcional desde Telegram. Extraer mensajes bajo demanda como módulo independiente para preparar futuros corpus, sin incorporarlos ni clasificarlos automáticamente en el experimento actual.	Diseño/desarrollo: 3.4, 4.4 y 4.8. Evidencia: extractor y sus tests. Evaluación: 5.1 confirma su exclusión.

Nota. RF = Requisito Funcional; la trazabilidad se limita a evidencias documentadas en el proyecto.

3.3 Requisitos no funcionales

La Tabla 3 resume los requisitos no funcionales de reproducibilidad, trazabilidad, robustez, mantenibilidad, separación de alcance y documentación. La última columna identifica su correspondencia con evidencias verificables y con la evaluación cuando resulta aplicable.

Tabla 3

Requisitos no funcionales del sistema experimental

Código	Descripción	Trazabilidad
RNF-1	Reproducibilidad. La ejecución debe poder repetirse a partir de los datos, la configuración y las condiciones documentadas.	Diseño/desarrollo: 3.7 y 4.7. Evidencia: configuración y manifiesto. Evaluación: 5.7 y 5.10.
RNF-2	Trazabilidad. Cada run debe tener un identificador único que agrupe configuración, métricas, predicciones, matriz de confusión y manifiesto.	Diseño/desarrollo: 3.7 y 4.6. Evidencia: run_id, manifiesto y artefactos. Evaluación: 5.1-5.5.
RNF-3	Robustez ante incoherencias del corpus. La validación debe detener el flujo cuando detecte incoherencias críticas.	Diseño/desarrollo: 3.6 y 4.2. Evidencia: validador y tests. Evaluación: 5.7.
RNF-4	Mantenibilidad y bajo acoplamiento. El diseño debe mantener separados los módulos de adquisición, preprocesamiento, entrenamiento y evaluación.	Diseño/desarrollo: 3.4 y 4.8. Evidencia: separación por módulos y tests unitarios; sin métrica específica.
RNF-5	Separación de Telegram respecto al experimento. La adquisición desde Telegram debe permanecer desacoplada del corpus etiquetado, el entrenamiento y la evaluación.	Diseño/desarrollo: 3.4 y 4.4. Evidencia: extractor desacoplado. Evaluación: 5.1.
RNF-6	Documentación e interpretabilidad. Las salidas deben ser comprensibles, auditables y aptas para el análisis académico.	Diseño/desarrollo: 3.7 y 4.6. Evidencia/evaluación: ejemplos de error y términos influyentes en 5.4 y 5.5.

Nota. RNF = Requisito No Funcional; la trazabilidad se limita a evidencias documentadas en el proyecto.

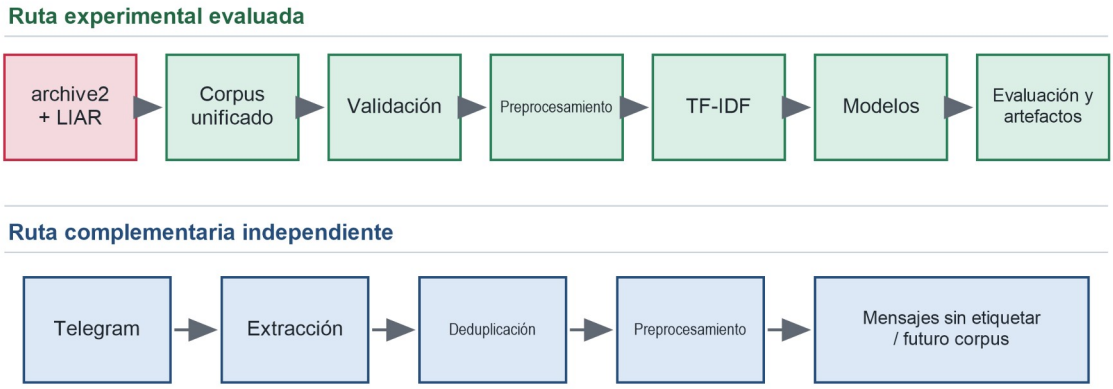
3.4 Arquitectura funcional

La arquitectura funcional se organiza en dos rutas independientes, representadas en la Figura 1. La ruta experimental evaluada parte de archive2 y LIAR, construye y valida el corpus unificado, aplica el preprocesamiento y la representación TF-IDF, entrena los modelos lineales y finaliza con la evaluación y la persistencia de artefactos.

Para facilitar la reproducibilidad, cada ejecución conserva la configuración, el manifiesto, las métricas, las predicciones, la matriz de confusión y los modelos. Estos elementos corresponden exclusivamente a la ruta experimental evaluada.

Figura 1

Arquitectura funcional con rutas experimental y complementaria independientes



Nota. Elaboración propia. Solo la ruta experimental evaluada sustenta los resultados experimentales presentados en esta memoria.

La ruta complementaria adquiere bajo demanda mensajes de canales públicos de Telegram, los deduplica y preprocesa, y produce mensajes sin etiquetar destinados únicamente a la preparación de un futuro corpus. Cualquier integración posterior con el flujo experimental requeriría definir un protocolo de selección, anotación y validación antes de emplear esos mensajes en el entrenamiento o la evaluación.

3.5 Herramientas y tecnologías

La Tabla 4 resume las tecnologías empleadas y su función dentro de este TFG.

Tabla 4

Herramientas y tecnologías utilizadas

Categoría	Herramienta	Función
Adquisición	Telethon	Acceso programático a canales públicos de Telegram.
Preprocesamiento	re, html, langdetect	Limpieza, tokenización, normalización y detección de idioma.
Datos	pandas, csv, zipfile	Construcción del corpus y lectura de fuentes externas.
Modelado	scikit-learn	TF-IDF, Regresión Logística, Linear SVC y métricas.
Persistencia	JSON (JavaScript Object Notation), CSV (valores separados por comas), joblib	Manifiestos, predicciones, modelos y resultados.
Pruebas	pytest	Validación automatizada de módulos y flujos principales.
Documentación	Word/PDF, APA7	Memoria final con tablas, figuras, citas y referencias.

Nota. TF-IDF = frecuencia de término–frecuencia inversa de documento (del inglés Term Frequency–Inverse Document Frequency).

3.6 Modelo de datos y gobierno del corpus

El modelo de datos del corpus se ha diseñado para conservar tanto información original como información derivada. La columna text mantiene el contenido de entrada, mientras que `normalized_text` almacena la versión procesada que alimenta el vectorizador. Esta separación evita que la limpieza borre evidencia útil para análisis posteriores. Si un error de clasificación depende de una URL sustituida, de una mención o de un hashtag normalizado, el investigador puede volver al texto original y comprobar qué información se perdió o se transformó.

`source_id` identifica cada fila original y permite enlazar la auditoría de exclusiones, el representante conservado, el grupo de duplicados y las predicciones fuera de entrenamiento. En el módulo independiente de Telegram, un identificador puede derivarse del canal y del mensaje, pero esos registros no entraron en los grupos experimentales. El orden de filas no se emplea como sustituto de esta trazabilidad.

Las columnas label y `label_name` se guardan de forma redundante. Aunque podría bastar con la etiqueta numérica, mantener el nombre de clase reduce errores de interpretación durante el análisis manual. La validación comprueba que ambos campos sean coherentes: 0 corresponde a `no_fake` y 1 corresponde a `fake_news`. Este control sencillo evita un fallo frecuente en experimentación, que consiste en invertir etiquetas sin advertirlo hasta que las métricas ya se han calculado.

`dataset_name` conserva la procedencia archive2 o LIAR de cada representante. Permite desglosar las métricas fuera de entrenamiento y definir las dos pruebas de transferencia; un grupo detectado con ambas procedencias se excluiría de esos entrenamientos y pruebas. Esta distinción muestra qué parte del rendimiento puede depender de estilo y formato de fuente, sin atribuir una causa única a cada error.

Los metadatos del corpus cumplen una función de gobierno documental. No se limitan a describir tamaños; registran archivos usados, archivos descartados, reglas de limpieza, etiquetas excluidas, distribución de idiomas, resumen de longitudes y hash del conjunto de datos. Esa información permite reconstruir por qué el corpus tiene su forma actual. También impone una disciplina: cuando se cambie una regla de construcción, deberá actualizarse el metadato y se generará un nuevo hash.

La Tabla 5 explicita el contrato de columnas del corpus derivado. Además de conservar los campos heredados de las fuentes, distingue el representante de los duplicados, el grupo de similitud y la procedencia combinada; estos campos permiten auditar la inclusión de cada fila original.

Tabla 5

Esquema canónico y función experimental de los campos

Campo	Contenido persistido	Función experimental
`source_id`	Identificador estable por fuente y registro.	Trazar duplicados, predicciones y reconstrucciones.
channel	Canal lógico o categoría de procedencia.	Conservar contexto sin usarlo como etiqueta.
`date_utc`	Fecha normalizada cuando la fuente la proporciona.	Documentar temporalidad disponible; puede estar vacía en LIAR.

Campo	Contenido persistido	Función experimental
text	Texto original combinado o declaración de origen.	Auditar transformaciones y ejemplos de error.
`normalized_text`	Texto tras limpieza determinista.	Entrada de TF-IDF y de los modelos lineales.
language	Idioma detectado por preprocesamiento.	Caracterizar el corpus y casos atípicos.
label	Clase binaria 0 o 1.	Objetivo supervisado del entrenamiento.
`label_name`	no_fake o fake_news en el corpus derivado; el nombre anterior solo identifica el baseline histórico.	Lectura humana y control de coherencia.
source / dataset_name	Fuente y colección integrada.	Analizar procedencia y posible sesgo.
`dataset_split`	full, train, valid o test según fuente.	Conservar partición original cuando existe.
`original_label`	Etiqueta antes de binarizar.	Auditar mapeos y exclusiones de LIAR.
title y topic	Título y categoría de origen cuando existen.	Reconstruir text y apoyar análisis descriptivos.
`duplicate_group`	Identificador del componente de textos exactos o casi duplicados.	Impedir que miembros de un grupo se distribuyan entre folds.
`historical_label_name`	Nombre de clase conservado del corpus anterior.	Mantener trazabilidad sin reinterpretarlo como etiqueta actual.
`group_sources`	Procedencias agregadas de todos los originales del grupo.	Excluir grupos mixtos de las transferencias entre fuentes.
`is_cross_source_group`	Indicador de procedencia mixta del grupo.	Auditar el aislamiento entre entrenamiento y prueba por fuente.

Nota. El esquema se cotejó con el encabezado del corpus derivado y con la auditoría por fila del run actual. Los campos opcionales conservan la ausencia de origen.

El contrato de datos no depende de la posición física de una fila en un archivo CSV. La identidad de procedencia permite volver al registro de origen y reconstruir por qué una entrada fue conservada, fusionada o excluida. Un representante del corpus derivado puede sintetizar varios originales idénticos sin borrar sus fuentes; la auditoría por fila retiene el vínculo, mientras que el corpus de entrenamiento contiene representantes y las variantes cercanas permitidas por el protocolo. Así se separa la unidad contable de la entrada original de la unidad utilizada en la partición experimental.

3.7 Reproducibilidad y control de cambios

La reproducibilidad se ha implementado en tres niveles. El primer nivel es la configuración, donde los parámetros de entrenamiento viven en un fichero JSON, no dispersos en instrucciones manuales. El segundo nivel es el versionado de artefactos, donde cada run crea una carpeta propia y no sobrescribe resultados anteriores. El tercer nivel es la validación automática, donde el pipeline no confía en que el conjunto de datos esté bien formado, sino que lo comprueba antes de entrenar.

La partición actual se genera mediante StratifiedGroupKFold con cinco *folds*, `shuffle` y `random_state` = 42. Cada componente de duplicados permanece en un único *fold*, y ambos clasificadores reciben exactamente las mismas asignaciones. El vocabulario TF-IDF y los pesos se ajustan exclusivamente con la fracción de entrenamiento de cada *fold*; el *fold* reservado produce las predicciones fuera de entrenamiento. Esta estrategia reduce una forma conocida de fuga por repetición, pero no sustituye una validación independiente en nuevos dominios.

Las huellas SHA-256 identifican los bytes de la fuente, la configuración, el código del ejecutor y el corpus derivado. El manifiesto registra además las versiones de Python, NumPy, pandas y scikit-learn, los tamaños de cada *fold*, la duración, las iteraciones, los avisos y las rutas de los artefactos. Reanudar una ejecución solo es válido si coinciden las identidades verificadas y se reutilizan etapas completadas; no se sobrescriben los resultados del *baseline* histórico.

Las tablas principales distinguen los artefactos del run de robustez —`robustness_manifest.json`, `exclusion_audit.csv`, `fold_assignments.csv`, predicciones y métricas por *fold* y transferencia— de los ficheros del *baseline* histórico, como `training_manifest.json` y `linear_model_terms.json`. Esta distinción permite rastrear cada cifra hasta su ejecución y evita combinar resultados de protocolos diferentes.

La trazabilidad relaciona cada conclusión con un identificador de ejecución, una huella de entrada y una partición. El nuevo run se denomina robustness-1a94400d502b149f; el SHA-256 de la fuente original es cfaf1b39c0260008e9328950649c9b83de2a9eadb478f631fb9cbc68d1b0a6b1 y el del corpus derivado es 184c707840a4bcff72dbef5e75e179152b630395c5f63f072db969bc1e79ea4e. Las salidas históricas mantienen sus propios manifiestos y no se mezclan con esta ejecución.

El control de cambios se aplica igualmente a las decisiones negativas. La exclusión de archive3, de las clases intermedias de LIAR y de mensajes de Telegram sin anotación forma parte de la definición experimental. Si cambian fuentes, reglas de construcción o dominio, se deberá conservar la configuración anterior y documentar una nueva evaluación, sin presentar el resultado como continuidad idéntica del *baseline* de referencia.

3.8 Decisiones de diseño y justificación técnica

La Tabla 6 resume las decisiones que definen el protocolo y sus cautelas. Los parámetros, artefactos y resultados de la ejecución actual se documentan en los capítulos 4 y 5; el Apéndice A conserva exclusivamente la referencia histórica.

Tabla 6

Decisiones de diseño y cautelas metodológicas

Decisión	Justificación principal	Limitación o cautela
Construir un corpus de dos fuentes	Permite contrastar documentos largos de archive2 con declaraciones breves de LIAR.	Las etiquetas, formatos y estilos de origen no son equivalentes.
Auditar duplicados antes de dividir	Reduce la posibilidad de que repeticiones exactas o casi duplicadas aparezcan en folds distintos.	MinHash LSH aproxima candidatos y puede omitir paráfrasis.
Mantener grupos en un solo fold	Evita repartir miembros de un componente de duplicados entre entrenamiento y evaluación.	La validación sigue circunscrita a dos fuentes públicas.
Representar texto con TF-IDF	Establece una línea base ligera y permite inspeccionar términos asociados.	No modela comprensión factual, contexto social ni multimodalidad.
Comparar dos modelos lineales	Aísla diferencias entre candidatos bajo la misma representación y particiones.	La selección es una comparación interna, no una estimación externa posterior.

Decisión	Justificación principal	Limitación o cautela
Fijar los umbrales antes de la evaluación	Mantiene la política configurada y evita optimizar sobre los folds observados.	Los valores 0,60 y 0,30 no son universales ni calibrados de igual forma.
Medir transferencia en ambas direcciones	Expone diferencias entre las fuentes al invertir entrenamiento y prueba.	Solo cubre archive2 y LIAR y excluye grupos con procedencia mixta.
Separar Telegram	Conserva como módulo adicional la adquisición bajo demanda sin introducir etiquetas inexistentes.	No se ha validado acceso real ni clasificación de mensajes de Telegram.

Nota. La tabla resume decisiones documentadas; no representa resultados adicionales.

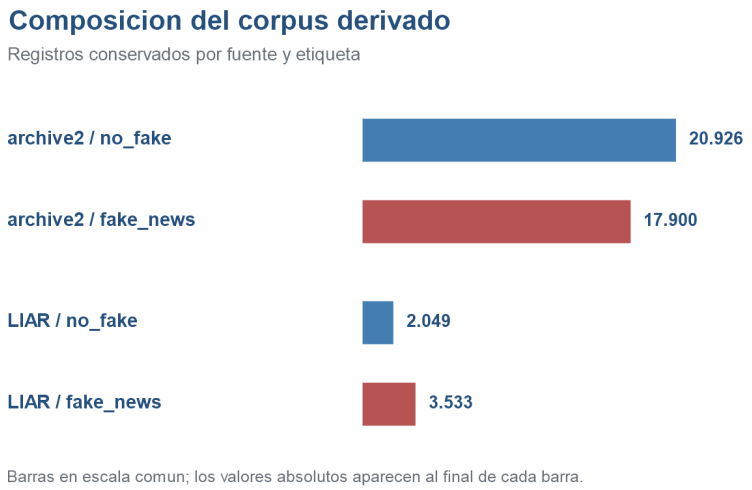
4. Desarrollo e implementación

4.1 Construcción del corpus derivado

La construcción histórica generó un archivo unificado de 50.503 filas: 44.898 proceden de archive2 y 5.605 de LIAR. La nueva preparación produce una versión derivada sin sobrescribir ese archivo, con auditoría por fila y grupos de duplicados. La Figura 2 representa únicamente los 44.408 registros conservados para la evaluación agrupada.

Figura 2

Composición del corpus derivado por fuente y etiqueta



Nota. Recuento de los representantes conservados en el corpus derivado del run robustness-1a94400d502b149f; las clases son etiquetas heredadas de las fuentes.

La Tabla 7 detalla las reglas de selección y transformación aplicadas a las fuentes originales.

Tabla 7

Criterios de inclusión y exclusión aplicados al corpus

Elemento	Tratamiento	Observación
archive2 / Fake.csv	Incluido: fake_news; 23.481 filas originales.	Etiqueta heredada del fichero; licencia no confirmada.
archive2 / True.csv	Incluido: no_fake; 21.417 filas originales.	No supone verificación independiente del registro.
LIAR / true	Incluida: no_fake; 2.053 declaraciones.	Se conserva original_label y procedencia.
LIAR / false	Incluida: fake_news; 2.505 declaraciones.	Equivalencia experimental, no semántica completa.
LIAR / pants-fire	Incluida: fake_news; 1.047 declaraciones.	La granularidad original se conserva en metadata.
LIAR half-true	Excluida: 2.627 filas.	Categoría intermedia de la escala original.
LIAR mostly-true	Excluida: 2.454 filas.	Categoría intermedia de la escala original.
LIAR barely-true	Excluida: 2.103 filas.	Categoría intermedia de la escala original.
Registros vacíos	Excluidos: 2 filas.	Regla del constructor; no se inventan etiquetas.
Duplicados por conflicto	Excluidos: 5 registros en 2 componentes.	Las etiquetas contradictorias no entran al experimento.

Nota. Las reglas proceden de fake_news_unified.metadata.json.

La Tabla 8 describe la composición del corpus derivado utilizado en el nuevo experimento.

Tabla 8

Composición auditada del corpus derivado

Característica	Valor auditado
Filas de entrada	50.503
Representantes conservados	44.408
Clase `no_fake`	22.975 (51,7%)
Clase `fake_news`	21.433 (48,3%)
archive2 conservado	38.826

Característica	Valor auditado
LIAR conservado	5.582
Duplicados exactos redundantes	6.090
Filas excluidas por conflicto	5
Componentes de casi duplicados agrupados	42; 45 aristas confirmadas
Componentes de procedencia mixta	0
SHA-256 del corpus derivado	184c707840a4bcff72dbef5e75e179152b630395c5f63f072db969bc1e79ea4e

Nota. Conteos mutuamente excluyentes de la auditoría por fila y el corpus derivado del run robustness-1a94400d502b149f. La etiqueta no_fake no equivale a verificación independiente.

La documentación local identifica en archive2 los ficheros `Fake.csv` y `True.csv` y conserva el identificador de descarga 2712039, que corresponde a la redistribución de emineyetm en Kaggle (2022). El material se asocia al conjunto original ISOT (Ahmed et al., 2018). La copia disponible no confirma la licencia de redistribución ni la procedencia detallada de cada registro; por ello no se atribuye una licencia.

El corpus es casi enteramente de lengua inglesa. Los metadatos del corpus registran la detección de en en 50.421 de los 50.503 registros. Las detecciones restantes son minoritarias y se interpretan con cautela como posible ruido de identificación automática o como efecto de textos breves, titulares o contenido parcialmente degradado; no acreditan un corpus multilingüe validado.

Las reglas de etiqueta conservan la fuente y el rótulo original. En archive2, Fake se mapea a `fake_news` y `True` a `no_fake`. En LIAR, `false` y `pants-fire` se mapean a `fake_news` y `true` a `no_fake`; se excluyen 2.627 `half-true`, 2.454 `mostly-true` y 2.103 `barely-true`. Dos registros sin texto útil se omiten en la construcción histórica. La clase `no_fake` es una etiqueta negativa heredada, no una verificación independiente de cada texto.

La metadata conserva recuentos finales, archivos utilizados, exclusiones, etiquetas originales, splits de LIAR, reglas de limpieza, longitudes de texto y distribución de idiomas. Esta información diferencia la disponibilidad de un CSV de una construcción documentalmente trazable. También permite comprobar que el predominio de archive2 no se oculte al interpretar las métricas posteriores.

La Tabla 9 concilia cada fila original con su estado final. Los 6.090 duplicados exactos son estados exclusivos de la auditoría; las 6.092 fusiones observadas durante una etapa intermedia no constituyen otro total final. La suma de representantes, duplicados y conflictos reproduce las 50.503 entradas.

Tabla 9

Conciliación final de filas originales por fuente

Fuente	Filas originales	Representantes	Duplicados exactos	Conflictos excluidos
archive2	44.898	38.826	6.070	2
LIAR	5.605	5.582	20	3
Total	50.503	44.408	6.090	5

Nota. Las categorías son mutuamente excluyentes y suman las filas originales. Los componentes conflictivos pueden incluir representantes y aliases exactos.

La construcción conserva las variantes no idénticas en un mismo grupo y descarta íntegramente los componentes con etiquetas contradictorias. La Tabla 10 ordena las fases sin confundir la preparación derivada con el *baseline* de una sola partición.

Tabla 10

Fases de construcción y contratos de salida

Fase	Entrada	Salida o invariante
Lectura archive2	Fake.csv y True.csv.	Procedencia y etiqueta binaria heredadas; incorporación posterior a la auditoría derivada.
Lectura LIAR	Particiones train, valid y test con etiqueta original.	Solo true, false y pants-fire pasan al mapeo binario.
Exclusión	Etiquetas intermedias y filas vacías.	No se fuerza una clase cuando el significado es intermedio o falta texto.
Normalización de esquema	Columnas heterogéneas de ambas fuentes.	Se emiten los campos del contrato canónico.
Identificación	Fuente, colección y registro de origen.	source_id permite rastreo y detección de duplicados.
Preprocesamiento	text y reglas declaradas.	normalized_text se deriva sin reemplazar el texto original.
Validación	Corpus integrado y metadata.	El entrenamiento exige esquema, etiquetas y texto consistentes.

Fase	Entrada	Salida o invariante
Persistencia	Dataset, metadata y manifiesto del run.	Corpus derivado, auditoría por fila y manifiesto de robustez vinculados a un run_id.

Nota. El flujo actual conserva un corpus derivado independiente y no sobrescribe el corpus histórico.

La búsqueda aproximada toma todas las secuencias contiguas de cinco palabras, sin recortar su inicio, centro o final. MinHash LSH propone candidatos; solo Jaccard sobre las secuencias completas confirma cada relación. Los componentes conexos se forman antes de descartar conflictos. La Tabla 11 fija los parámetros y sus límites.

Tabla 11

Parámetros de agrupación y límites de la deduplicación

Decisión	Configuración	Efecto y límite
Duplicado exacto	Texto normalizado idéntico; representante con menor source_id.	Se conserva la procedencia de cada fila en la auditoría.
Candidatos aproximados	Shingles contiguos de 5 palabras; 128 permutaciones; semilla 42; LSH 32 bandas × 4 filas.	Las firmas se procesan en lotes de 256 filas.
Confirmación	Jaccard ≥ 0,90 sobre todas las secuencias completas.	No se recortan shingles al inicio, centro o final.
Grupos	Componentes conexos; variantes cercanas retenidas bajo el mismo duplicate_group.	Los componentes se forman antes de excluir conflictos.
Etiquetas contradictorias	Excluir componentes con más de una etiqueta original incompatible.	2 componentes; 5 filas en estados de auditoría excluidos.
Transferencia	Excluir grupos cuya procedencia combinada contenga ambas fuentes.	No se detectaron grupos mixtos en esta ejecución.

Nota. MinHash LSH reduce el espacio de búsqueda, pero no garantiza descubrir todas las paráfrasis ni los casi duplicados que no compartan banda.

La conciliación exige distinguir recuentos que responden a preguntas diferentes. Las entradas originales describen el material recibido de dos fuentes; los representantes describen los registros disponibles tras consolidar coincidencias exactas y excluir conflictos; las aristas confirmadas describen relaciones entre textos cercanos, no filas eliminadas. Por tanto, las 45 aristas de 42 componentes no se restan de los 44.408 representantes. Las variantes no idénticas quedan bajo un identificador de grupo y se particionan como unidad. Eliminarlas todas alteraría el corpus y constituiría otro protocolo.

La equivalencia experimental entre `false` y `pants-fire` de LIAR define la clase positiva de este estudio, pero no afirma que ambas categorías tengan idéntica intensidad semántica. `no_fake` reúne etiquetas negativas heredadas de archive2 y LIAR sin significar verificación factual independiente. Las categorías intermedias se excluyen para evitar una binarización que el origen no respalda. `original_label` permite localizar las entradas afectadas si un trabajo futuro revisa el mapeo. La procedencia se conserva como metadato de análisis, no como rasgo textual explícito del vectorizador.

4.2 Validación del corpus

La validación estructural comprueba columnas, tipos, etiquetas, integridad, nulos y registros duplicados antes de entrenar; es distinta de la validación semántica, que requeriría juzgar de forma independiente la veracidad y la calidad de las etiquetas. El constructor produce un registro por cada fila original con estado retained_representative, deduplicated_exact o excluded_conflicting_duplicate_group, además de representante y grupo. Los conteos exclusivos (44.408 + 6.090 + 5) reconstruyen las 50.503 filas de entrada.

Los controles del nuevo ejecutor verifican columnas exigidas, `source_id` únicos, parámetros del protocolo y grupos no repartidos entre entrenamiento y prueba; los tests sintéticos cubren esas invariantes. El validador `scripts/validate_dataset.py` conserva los controles estructurales del corpus histórico. Ninguno de estos mecanismos comprueba por sí mismo la verdad factual, la calidad semántica de las etiquetas ni la representatividad de las fuentes.

En la versión derivada, cada fila de entrada queda identificada como representante conservado, duplicado exacto o exclusión por componente contradictorio. Los duplicados exactos comparten el representante de menor `source_id`; los casi duplicados confirmados permanecen bajo un mismo duplicate_group. El ejecutor rechaza columnas ausentes e identificadores

`source_id` repetidos antes de formar particiones. Los controles estructurales y sus límites se detallan en la Tabla 12.

La preparación determina candidatos mediante MinHash LSH de 128 permutaciones, semilla 42 y 32 bandas de cuatro filas sobre todas las secuencias de cinco palabras; el Jaccard de las secuencias completas confirma pares con similitud de al menos 0,90. Se procesan lotes de 256 documentos para limitar memoria. Los componentes se forman antes de excluir aquellos con etiquetas incompatibles. LSH aproxima candidatos y no garantiza descubrir todas las paráfrasis.

La Tabla 12 distingue controles estructurales de calidad semántica. Un esquema válido y una etiqueta binaria coherente permiten ejecutar el protocolo, pero no verifican la verdad de una noticia ni la representatividad de la fuente.

Tabla 12

Controles del corpus y límites de interpretación

Control	Condición	Acción	Límite
Esquema	Están las columnas obligatorias.	Un campo ausente se informa como error.	La corrección factual del contenido.
Identidad	source_id no contiene duplicados.	Evita contar o predecir varias veces el mismo registro.	No certifica calidad factual ni generalización.
Existencia	El dataset no está vacío.	Un corpus vacío no puede entrenar ni evaluar.	No certifica calidad factual ni generalización.
Etiquetas	label pertenece a {0, 1}.	Preserva la formulación binaria documentada.	Que la etiqueta externa sea verdadera en sentido absoluto.
Coherencia	label y label_name describen la misma clase.	Evita invertir la semántica de las clases.	No certifica calidad factual ni generalización.
Nulos	No hay nulos en columnas requeridas.	Detiene el flujo antes de datos incompletos.	No certifica calidad factual ni generalización.
Texto	text y normalized_text no están vacíos.	Evita vectorizar registros sin contenido.	No certifica calidad factual ni generalización.
Advertencias	Se detecta una única clase o idioma cuando procede.	Exige revisión; no equivale a validación semántica.	No certifica calidad factual ni generalización.

Control	Condición	Acción	Límite
Duplicación textual	Se comparan textos normalizados antes de particionar.	Agrupar variantes cercanas y registrar exclusiones conflictivas.	No demuestra ausencia de todas las paráfrasis.
Idioma	El detector usa semilla fija y persiste language.	Registrar detecciones minoritarias y revisar casos atípicos.	No acredita validación multilingüe.
Banderas de calidad	Se guardan quality_flags e is_usable.	Separar advertencias técnicas de errores bloqueantes.	No decide la veracidad de un contenido.

Nota. Los controles estructurales proceden del validador y del ejecutor; la validación semántica requiere evidencia independiente.

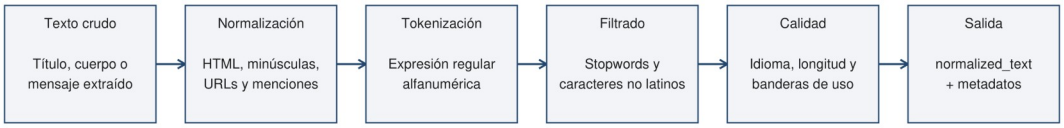
La verificación de duplicados tiene dos niveles. El validador estructural rechaza `source_id` repetidos en la entrada de entrenamiento; la preparación derivada analiza textos normalizados antes de particionar y registra los originales de cada representante. Repetición de clave y coincidencia textual no son lo mismo. El primer control protege la integridad de identidades; el segundo reduce solapamiento entre prueba y entrenamiento. Ninguno demuestra ausencia de todas las paráfrasis, porque MinHash LSH solo aproxima la propuesta de candidatos.

4.3 Preprocesamiento textual

El módulo *`preprocessing.py`* transforma cada texto crudo en una representación normalizada. La Figura 3 muestra las etapas del pipeline. La normalización conserva trazabilidad, ya que no sustituye los campos originales por los derivados. El texto crudo, la fuente, el identificador y la etiqueta permanecen disponibles para auditoría.

Figura 3

Etapas del pipeline de preprocesamiento textual



La limpieza no borra la trazabilidad. El registro conserva la fuente, el identificador, la etiqueta original y los campos derivados necesarios para reproducir el entrenamiento.

Nota. Elaboración propia a partir de las reglas persistidas en fake_news_unified.metadata.json.

La Tabla 13 describe las reglas de limpieza aplicadas.

Tabla 13

Reglas de preprocesamiento documentadas

Regla	Transformación	Evidencia conservada
R1	Convierte entidades HTML y normaliza el texto a minúsculas.	text se mantiene y normalized_text registra el resultado.
R2	Sustituye URLs por el token 'url' y menciones por el token 'usuario'.	Los marcadores hacen visible la regla aplicada.
R3	Conserva el término base de los hashtags y elimina el símbolo '#'.	Los tokens pueden revisarse en la representación derivada.
R4	Elimina emojis y caracteres fuera de la clase latina configurada.	La regla queda declarada en cleaning_rules.
R5	Compacta espacios en blanco y tokeniza con una expresión regular alfanumérica.	Se persisten tokens y token_count en la salida.
R6	Filtra stopwords configurables por entorno antes de reconstruir normalized_text.	Evita asumir una lista fija no documentada.
R7	Marca textos vacíos, demasiado cortos o con idioma desconocido mediante quality_flags.	quality_flags e is_usable conservan información diagnóstica.

Nota. Las reglas quedan persistidas junto al corpus para que la transformación pueda revisarse sin abrir el código fuente.

La normalización se ejecuta en un orden estable: descodifica HTML, pasa el texto a minúsculas, sustituye URLs y menciones por marcadores, conserva el término base de los hashtags, elimina caracteres fuera de la clase configurada y compacta los espacios antes de tokenizar. El resultado no borra text, sino que se almacena junto a él. Esta decisión permite revisar qué transformación se aplicó a un registro concreto y evita que una representación adecuada para TF-IDF se confunda con la fuente original.

El detector de idioma utiliza una semilla fija. Junto al idioma se guardan tokens, recuento de tokens, `quality_flags` e `is_usable`. Las banderas no corrigen por sí mismas un texto, pero convierten condiciones como vacío, demasiado corto o idioma desconocido en información visible. En el corpus de referencia, las detecciones distintas de en son minoritarias y no deben interpretarse como validación multilingüe.

El orden de la limpieza es determinista y el texto original se conserva junto con la representación normalizada. La semilla fija del detector mejora la repetibilidad, pero no certifica el idioma; las banderas de calidad tampoco sustituyen una anotación semántica. Una modificación de *stopwords*, tokenización o limpieza define nuevas condiciones experimentales.

4.4 Módulo complementario de adquisición desde Telegram

La ruta experimental descrita en esta memoria utiliza exclusivamente el corpus binario etiquetado construido a partir de archive2 y LIAR. Los mensajes adquiridos desde Telegram no se incorporaron a ese corpus ni participaron en el entrenamiento, la comparación de modelos o la evaluación. Por tanto, los resultados del capítulo 5 corresponden únicamente a los conjuntos de datos públicos y a la partición allí documentada.

`telegram_extractor.py` implementa, como capacidad técnica complementaria, la adquisición bajo demanda de mensajes procedentes de canales configurados mediante Telethon. Las credenciales se obtienen del entorno, la resolución de canales admite reintentos y los fallos quedan registrados en un informe. Para cada mensaje con texto se conservan el identificador del mensaje, el canal y su identificador, el remitente, la fecha en tiempo universal coordinado (UTC) y el contenido. Esta fase no asigna etiquetas ni genera predicciones.

Tras la adquisición, *`pipeline.py`* coordina la deduplicación persistente de *`dataset_manager.py`* mediante el identificador compuesto *`canal:message_id`* y aplica la normalización definida en *`preprocessing.py`*. El proceso persiste la colección cruda, los mensajes procesados, el registro de identificadores y el manifiesto de ejecución. La salida permanece sin etiquetar y constituye únicamente material candidato para construir en el futuro un corpus específico del dominio de Telegram.

Antes de incorporar esos mensajes a un experimento supervisado sería necesario establecer criterios de selección, un protocolo de anotación, controles de calidad y acuerdo entre anotadores, además de una partición y un protocolo de evaluación específicos del dominio. La implementación actual acredita una capacidad de adquisición bajo demanda, no una

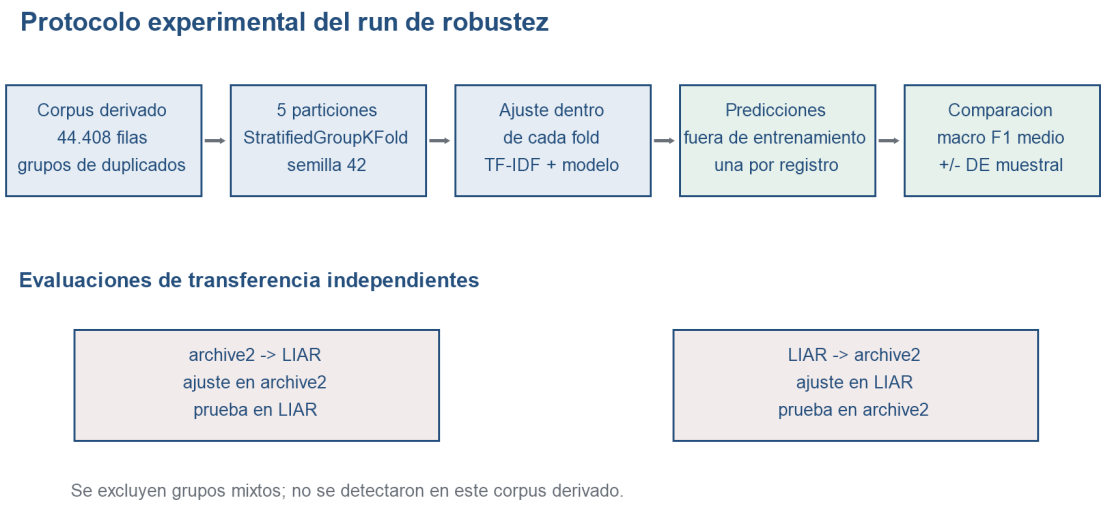
monitorización continua, un despliegue productivo ni una validación del clasificador sobre canales o conversaciones reales.

4.5 Configuración de los experimentos

La ejecución nueva usa `configs/robustness_config.json` y no modifica el comportamiento del entrenamiento histórico. El mismo vectorizador TF-IDF de unigramas y bigramas se integra con regresión logística y Linear SVC; cada pipeline aprende su vocabulario exclusivamente a partir de la partición de entrenamiento correspondiente. La Figura 4 representa las cinco particiones agrupadas y las dos direcciones de transferencia entre fuentes.

Figura 4

Validación cruzada agrupada y evaluación de transferencia entre fuentes



Nota. Elaboración propia a partir de configs/robustness_config.json y del manifiesto de la ejecución de robustez; cada grupo permanece dentro de una única partición.

La configuración utiliza cinco particiones estratificadas y agrupadas, semilla 42 y asignaciones idénticas para los dos modelos. La regresión logística conserva C = 1, `class_weight` = balanced, `max_iter` = 1.000 y solver `lbfgs`; Linear SVC conserva C = 1, `class_weight` = balanced y dual = True, con `liblinear` como solver. Los *folds* reservados generan predicciones fuera de entrenamiento para todos los registros.

La validación cruzada estratificada y agrupada usa cinco particiones, `shuffle` y semilla 42. Las asignaciones se comparten entre ambos modelos. En cada *fold*, el vectorizador y el

clasificador se ajustan solo con los registros de entrenamiento; cada ejemplo recibe una predicción fuera de entrenamiento. Se guardan métricas por *fold*, media y desviación estándar muestral, matrices globales y desglose por fuente. Se registran duración, número de iteraciones y avisos de convergencia; no se atribuyen retrasos al solver sin evidencia medida.

La política de decisión también es parte de la configuración. El valor 0,60 se aplica a la salida probabilística de Regresión Logística y 0,30 a `decision_function` de Linear SVC. Ambos valores son manuales y no fueron aprendidos durante el ajuste. Su presencia en el manifiesto permite distinguir la métrica del modelo de una política de umbral que podría cambiar en otro escenario y exigir una evaluación nueva.

La Tabla 14 reúne la configuración efectiva del run principal. Los umbrales permanecieron fijados antes de observar las nuevas evaluaciones; 0,30 es una puntuación de decisión de Linear SVC y no una probabilidad.

Tabla 14

Configuración efectiva de la evaluación agrupada

Elemento	Valor configurado	Función o cautela
Ejecución	robustness-1a94400d502b149f	Identidad del experimento actual.
Particiones	5; StratifiedGroupKFold; semilla 42	Grupos íntegros y folds idénticos para ambos modelos.
Representación	TF-IDF; unigramas y bigramas; max_features = 5000	Ajuste exclusivo con entrenamiento de cada fold.
Regresión logística	C = 1; class_weight = balanced; solver = lbfgs	Umbral probabilístico fijo 0,60.
Linear SVC	C = 1; class_weight = balanced; dual = True	Umbral 0,30 sobre decision_function.
Selección	Macro F1 medio de cinco folds	Comparación interna, no evaluación independiente posterior.
Transferencia	archive2 → LIAR y LIAR → archive2	Exclusión previa de grupos de procedencia mixta.

Nota. Configuración cotejada con configs/robustness_config.json y el manifiesto del run actual; no se modificaron los parámetros del baseline histórico.

La validación cruzada y la transferencia responden a preguntas distintas. En la primera, los cinco *folds* se forman sobre el corpus derivado y cada componente de duplicados permanece íntegro en una sola partición; los modelos comparten asignaciones para que la comparación no dependa de pruebas diferentes. En la segunda, un modelo se ajusta con una fuente y se prueba en la otra, excluyendo antes cualquier grupo de procedencia mixta. No se trata de un sexto *fold* ni de una búsqueda de umbrales: se examina el cambio conjunto de procedencia, longitud, estilo y etiquetas. El protocolo no permite atribuir la diferencia a un solo factor causal.

Cada ajuste vectoriza exclusivamente textos de entrenamiento. Esta restricción evita que el vocabulario y los pesos IDF incorporen documentos de prueba, aunque no elimina sesgos de fuente. Los umbrales 0,60 y 0,30 se mantienen tal como se declararon; el primero actúa sobre probabilidades estimadas y el segundo sobre `decision_function`. Sus escalas no son equivalentes y sus valores numéricos no deben leerse como una misma medida de confianza. El candidato se elige por el macro F1 medio de los cinco *folds*, nunca por el resultado aislado más favorable.

4.6 Artefactos de trazabilidad

La ejecución de robustez tiene un directorio propio y conserva `robustness_manifest.json`, `protocol.json`, `preparation.json`, `exclusion_audit.csv`, `fold_assignments.csv`, predicciones fuera de entrenamiento, métricas por *fold*, métricas de transferencia y modelos serializados. El manifiesto incorpora identificadores, huellas SHA-256, versiones de dependencias, parámetros, duración, iteraciones y estado por etapa. El *baseline* de abril de 2026 mantiene intactos sus artefactos y se trata como referencia histórica independiente. La Figura 5 muestra, como referencia documental separada, un fragmento del manifiesto del *baseline* histórico.

Figura 5

Ejemplo del manifiesto del baseline histórico

training_manifest.json - extracto del run reproducible


```
{
    "run_id": "baseline-20260411T145313Z-a9ea359a",
    "dataset_sha256": "cfaf1b39c0260008e9328950...",
    "text_column": "normalized_text",
    "label_column": "label",
    "best_model": "linear_svc",
    "selection_metric": "macro_f1",
    "train_rows": 37877,
    "holdout_rows": 12626,
      "probability_threshold": 0.6,
      "decision_threshold": 0.3
}
```


La captura muestra los campos mínimos que conectan configuración, dataset, modelo y evaluación.

Nota. Captura generada a partir del fichero training_manifest.json del run baseline-20260411T145313Z-a9ea359a.

La Tabla 15 distingue los artefactos de la ejecución agrupada de los ficheros conservados del *baseline* histórico.

Tabla 15

Artefactos persistidos por ejecución

Artefacto	Contenido	Uso en la memoria
training_manifest.json	Configuración, rutas, hash del dataset y métricas agregadas.	Evidencia del baseline histórico; no resume el nuevo run.
comparison.json	Resultados de todos los modelos candidatos.	Evidencia del baseline histórico; no resume el nuevo run.
classification_report.json	Precisión, recall y F1 por clase.	Evidencia del baseline histórico; no resume el nuevo run.
confusion_matrix.json	Matriz de confusión estructurada.	Evidencia del baseline histórico; no resume el nuevo run.

Artefacto	Contenido	Uso en la memoria
holdout_predictions.csv	Predicción de cada ejemplo del conjunto de validación.	Evidencia del baseline histórico; no resume el nuevo run.
prediction_examples.json	Ejemplos representativos de aciertos y errores.	Evidencia del baseline histórico; no resume el nuevo run.
linear_model_terms.json	Términos con mayor peso por clase.	Evidencia del baseline histórico; no resume el nuevo run.
robustness_manifest.json	Estado, versiones, identidad y métricas del run nuevo.	Vincular resultados al run actual.
fold_assignments.csv	Asignación de cada source_id a uno de cinco folds.	Auditar separación de grupos.
*_oof_predictions.csv	Una predicción fuera de entrenamiento por registro y modelo.	Matriz global y análisis por fuente.
*_fold_*.json	Métricas, umbral, duración, iteraciones y avisos de cada ajuste.	Comparar folds sin recalcular cifras.
transfer_*.json y modelos	Predicciones, matrices, métricas y modelos para las cuatro transferencias.	Documentar el resultado en ambas direcciones.
preparation.json y protocol.json	Identidad de preparación y protocolo documentado antes de la ejecución.	Auditar decisiones y estados de la preparación.
exclusion_audit.csv	Estado y representante de cada fila original.	Conciliar inclusiones, duplicados y conflictos.
pytest_junit.xml	Resultados ejecutados de la suite actual.	Sustentar las 28 pruebas superadas.

En el nuevo protocolo, `robustness_manifest.json` enlaza la configuración y las huellas con las etapas completadas; `exclusion_audit.csv` y `fold_assignments.csv` permiten comprobar qué filas y grupos se evaluaron; los CSV de predicciones y los JSON de métricas, junto con los modelos, documentan cada ajuste. Los ficheros `comparison.json`, `classification_report.json`, `confusion_matrix.json`, `holdout_predictions.csv`, `prediction_examples.json` y `linear_model_terms.json` pertenecen al *baseline* histórico y no se utilizan para calcular los resultados principales.

La disponibilidad de un archivo no demuestra por sí sola que un modelo haya sido evaluado. El manifiesto del run vincula configuración, versiones, hashes, asignaciones, modelos, predicciones y métricas; la auditoría comprueba esa correspondencia sin atribuir al experimento actual los términos o predicciones del *baseline* histórico.

La Tabla 16 fija las huellas criptográficas de la ejecución. Identifican bytes concretos, no validez semántica; un cambio de código, configuración o datos invalida la reutilización automática de una etapa completada.

Tabla 16

Identidad criptográfica de la ejecución principal

Objeto	SHA-256
`run_id`	robustness-1a94400d502b149f
Fuente de entrada	cfaf1b39c0260008e9328950649c9b83de2a9eadb478f631fb9cbc68d1b0a6b1
Configuración	98ab0ed4887bca0fa5274bd782ff32776c5dfd638b79eed882b1917a8a81787e
Ejecutor	83569de79e512e50957d65babdfe82379888b4b51cbd7cec79cba58950e934e8
Corpus derivado	184c707840a4bcff72dbef5e75e179152b630395c5f63f072db969bc1e79ea4e
Auditoría por fila	604ce0fb11703f6120f25c2ebd5a38572a0c53f113ab4467312837df0afd5ad8

Nota. Las huellas se leen de robustness_manifest.json y preparation.json; identifican bytes concretos, no calidad semántica.

4.7 Protocolo de reproducción

El protocolo de reproducción se estructura en pasos ordenados, porque cada uno consume artefactos del anterior. La Tabla 17 resume el flujo mínimo. Cuando cambian el conjunto de datos o la configuración, el run anterior se conserva y se genera uno nuevo, ya que la comparación de resultados bajo condiciones distintas requiere mantener ambos registros.

Tabla 17

Protocolo mínimo para reproducir el experimento

Fase	Comando o etapa	Resultado persistido
Identidad	run_robustness_experiments.py --dry-run	Identidad, config y hashes previstos.
Preparación	Ejecución con configs/robustness_config.json	Corpus derivado, auditoría fila a fila y grupos.
Validación cruzada	Cinco folds agrupados y compartidos	Modelos, métricas y predicciones OOF por fold.
Transferencia	archive2 -> LIAR y LIAR -> archive2	Cuatro modelos, salidas y métricas.
Recuperación	--resume <run_id>	Reutiliza etapas completas solo si coinciden los hashes.
Pruebas	pytest -q	Informe JUnit del estado actual de la suite.

La reproducción auditable exige identificar la entrada exacta, los parámetros, las particiones compartidas, el ajuste exclusivo en entrenamiento y los artefactos persistidos. La reanudación reutiliza solo etapas completas cuyas huellas coinciden. Los modelos de transferencia se conservan por dirección y candidato, sin mezclar sus conjuntos de prueba.

La identidad de una ejecución no se reduce al nombre del directorio. El manifiesto incorpora hashes de entrada, configuración efectiva, ejecutor y corpus derivado, además de versiones y estado por etapa. La reanudación reutiliza un ajuste completo solo cuando coinciden las identidades requeridas; una etapa interrumpida no se convierte en resultado final por haber producido archivos parciales. Esta regla protege la correspondencia entre métricas, modelos y predicciones, pero no certifica la calidad de las etiquetas. La auditoría distingue así integridad técnica, repetibilidad y validez externa.

4.8 Implementación por módulos

La implementación se ha dividido en módulos pequeños, cada uno con una responsabilidad clara. Esta organización facilita probar partes concretas del sistema y evita que la lógica de entrenamiento se mezcle con la adquisición de datos. Aunque el proyecto no persigue una arquitectura de microservicios, sí aplica separación funcional: extraer, normalizar, construir, validar, entrenar, evaluar y analizar son actividades distintas.

telegram_extractor.py encapsula la comunicación con Telegram mediante Telethon y actúa como adaptador de entrada independiente. No decide etiquetas, no aplica entrenamiento ni clasifica mensajes; su salida contiene únicamente texto y metadatos mínimos. Esa colección no forma parte del corpus etiquetado de archive2 y LIAR utilizado en el experimento. Si en un trabajo futuro cambiara la fuente de adquisición, el flujo de preparación podría mantenerse siempre que se entregara un fichero compatible.

dataset_manager.py gestiona la deduplicación persistente. La deduplicación no se ha implementado como un filtrado puntual en memoria, porque las sesiones de adquisición pueden repetirse en días distintos. Mantener un registro de identificadores ya vistos evita que el mismo mensaje se incorpore varias veces a la colección cruda. Además, permite una adquisición incremental sin confundir esa colección sin etiquetar con el corpus experimental actual; su conversión en un corpus de dominio exigiría el protocolo de selección y anotación descrito en el apartado 4.4.

preprocessing.py contiene la lógica de limpieza textual. El módulo devuelve un objeto con texto normalizado, tokens, recuento de tokens, idioma detectado, banderas de calidad e indicador de usabilidad. Esta salida estructurada resulta preferible a devolver solo una cadena, porque permite conservar información diagnóstica. Un texto demasiado corto o con idioma desconocido no se confunde con un texto normalizado válido; queda marcado para que las fases siguientes decidan cómo tratarlo.

scripts/build_real_dataset.py integra las fuentes externas. Su responsabilidad no es entrenar, sino construir el corpus canónico y escribir metadatos. Esta separación evita que el entrenamiento tenga que conocer detalles internos de archive2 o LIAR. Si se añadiera una tercera fuente, el cambio debería concentrarse en la construcción del conjunto de datos y en la documentación de reglas, no en el clasificador.

El script `scripts/validate_dataset.py` documenta el contrato estructural del corpus histórico. El ejecutor del protocolo agrupado aplica controles propios de columnas, identificadores, configuración, grupos y asignaciones antes de ajustar los modelos. Así se distinguen las verificaciones de la primera versión y las efectivamente aplicadas a los resultados nuevos; ninguno de ambos controles verifica la verdad factual de las etiquetas.

El ejecutor `scripts/run_robustness_experiments.py` coordina la preparación derivada, la validación cruzada agrupada, la transferencia y la persistencia recuperable. Cada partición construye un pipeline de TF-IDF y modelo que ajusta el vocabulario exclusivamente con su subconjunto de entrenamiento. Los modelos serializados quedan separados por candidato, partición y dirección de transferencia. El entrenamiento histórico permanece sin cambios.

`analysis_utils.py` agrupa funciones de análisis complementario. Su papel es generar matriz de confusión, ejemplos representativos y términos influyentes. Mantener estas tareas fuera del script principal reduce complejidad y permite probarlas por separado. También facilita que la memoria use artefactos ya generados en lugar de recalcular manualmente estadísticas durante la redacción.

`scripts/evaluate_baseline.py` permite evaluar un modelo persistido a partir de su manifiesto. Esta decisión es útil cuando se quiere comprobar que el modelo guardado y los artefactos siguen siendo coherentes. También permite distinguir dos escenarios: entrenamiento de un nuevo *baseline* y evaluación de un *baseline* ya existente. En trabajos académicos, esa distinción evita confundir ajuste de modelo con verificación posterior.

Las interfaces distinguen construcción de datos, validación, preprocesamiento, ejecución, evaluación y exportación. El manifiesto funciona como punto de enlace; las asignaciones de *fold* y predicciones se pueden contrastar por `source_id` y duplicate_group. El adaptador de Telegram continúa desacoplado y no conoce el vectorizador ni los modelos del experimento evaluado.

La separación funcional facilita pruebas unitarias con entradas sintéticas. La suite actual cubre preparación, conflictos, agrupación, particiones, aislamiento del entrenamiento y recuperación; sus 28 casos superados verifican contratos técnicos, no estiman rendimiento estadístico. La validez experimental se estudia mediante la validación cruzada agrupada y las dos transferencias entre archive2 y LIAR, con los límites de dominio descritos en el capítulo 5.

4.9 Pruebas y control de calidad

La suite se ejecutó en el entorno local de Python 3.11.9 tras añadir pruebas específicas del protocolo. El informe actual registra 28 pruebas superadas y ninguna fallida; el resultado histórico de 16 pruebas pertenece a una versión anterior y no se reutiliza como evidencia nueva. El XML de pytest se conserva dentro del directorio del run para distinguir la validación actual del inventario anterior.

Las pruebas de construcción del conjunto de datos verifican que el mapeo de etiquetas y los descartes se apliquen de forma consistente. En una tarea de *fake news*, un error en el mapeo `true`, `false` o `pants-fire` cambiaría el significado completo del experimento. Por ello, esas reglas se codifican y se cubren mediante pruebas, además de documentarse en la memoria.

Las pruebas de entrenamiento comprueban que un conjunto de datos sintético mínimo produzca un manifiesto y artefactos esperados. Aunque ese conjunto de datos no mida rendimiento real, sí valida la integración entre configuración, entrenamiento, métricas y persistencia. Si una refactorización rompe la generación de `comparison.json` o `confusion_matrix.json`, el fallo se detecta antes de redactar nuevas conclusiones.

Las pruebas del módulo de Telegram siguen empleando el cliente simulado `_FakeClient` y cubren la forma de la salida y las condiciones de configuración. El informe actual no prueba credenciales, acceso real, reintentos de red, funcionamiento continuo ni eficacia del clasificador en Telegram.

El control de calidad documental se apoya en la coherencia entre artefactos y texto. Cada cifra incorporada a la memoria procede de un JSON o CSV concreto. Este criterio ha guiado la ampliación del documento. Las figuras de distribución, métricas, matriz de confusión, términos influyentes y manifiesto se han generado desde los artefactos del proyecto. La memoria no introduce capturas decorativas, sino evidencias vinculadas a lo que se explica.

La documentación de calidad distingue corrección técnica y validez experimental. Las pruebas automatizadas comprueban transformaciones, construcción, persistencia e interfaces mediante entradas controladas. Las cinco particiones agrupadas y las dos evaluaciones entre fuentes aportan la evidencia cuantitativa de esta versión; el *holdout* del *baseline* se conserva solo como referencia histórica. Ninguna de esas pruebas acredita clasificación factual de mensajes de Telegram ni generalización universal.

El informe JUnit actual registra 28 casos correctos: 16 pruebas del conjunto anterior y 12 del módulo de robustez. La Tabla 18 delimita qué se comprueba y qué permanece fuera de alcance; el resultado de 16 pruebas citado en versiones anteriores es histórico, no una ejecución adicional del protocolo actual.

Tabla 18

Cobertura de las 28 pruebas automatizadas

Grupo	n	Evidencia cubierta	Límite explícito
Preprocesamiento	4	Limpieza, tokens, idioma y forma de salida.	No mide calidad lingüística ni rendimiento.
Corpus y validación	3	Construcción, mapeos, exclusiones y metadata sintética.	No certifica etiquetas externas ni representatividad.
Telegram simulado	3	Estructura, canal e informe mediante _FakeClient.	No valida acceso real, credenciales ni rendimiento.
Entrenamiento y configuración	2	Manifiesto, artefactos y umbrales en datos sintéticos.	No recalcula métricas del corpus real.
Evaluación y persistencia	1	Resumen y archivos de un modelo de prueba.	No demuestra transferencia ni validez estadística.
Análisis	3	Ejemplos priorizados, términos y política de decisión.	No convierte términos en causas.
Robustez agrupada	12	Etiquetas, deduplicación, conflictos, grupos, folds, aislamiento y persistencia.	No prueba generalización a otros dominios.

Nota. n = número de pruebas. Informe actual: reports/robustness_runs/robustness-1a94400d502b149f/pytest_junit.xml. Las 16 anteriores y las 12 de robustez forman la misma ejecución de 28 pruebas sin fallos.

5. Evaluación experimental y análisis de resultados

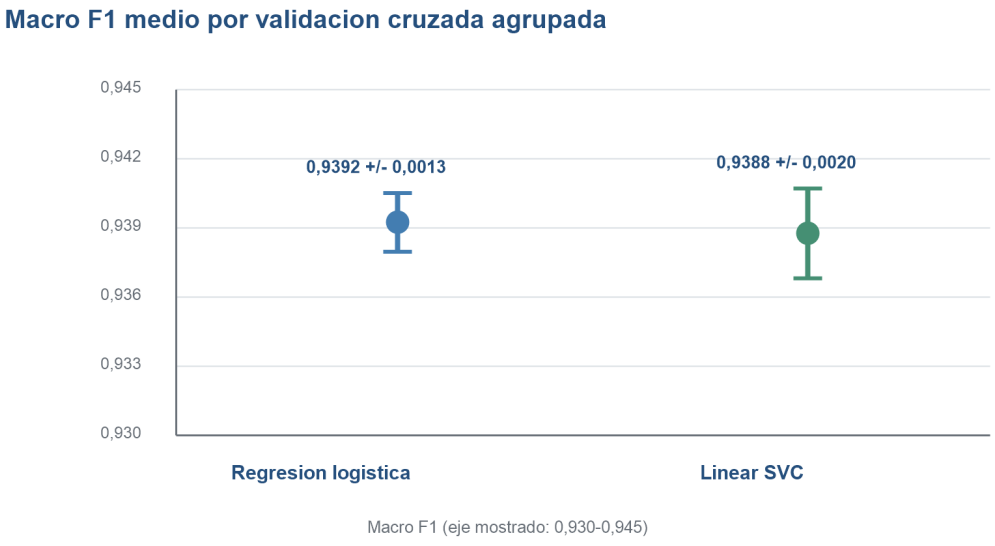
La evaluación principal utiliza la versión derivada de 44.408 registros: 22.975 `no_fake` y 21.433 `fake_news`, procedentes exclusivamente de archive2 y LIAR. Se emplea StratifiedGroupKFold con cinco *folds*; ambos modelos comparten las asignaciones y cada predicción se genera con un vectorizador ajustado solo sobre entrenamiento. Telegram queda excluido. Los resultados se limitan a este corpus y a esta comparación interna; las transferencias entre fuentes se presentan por separado y no implican validez en plataformas no estudiadas.

5.1 Comparativa de modelos candidatos

La Figura 6 y la Tabla 19 presentan la media y la desviación estándar muestral de las cinco particiones para los dos modelos. La regresión logística obtuvo el mayor macro F1 medio (0,9392 ± 0,0013), frente a 0,9388 ± 0,0020 de Linear SVC. La diferencia es pequeña, se utiliza únicamente como criterio interno de selección y no se interpreta como una diferencia estadísticamente significativa ni como estimación independiente posterior a la selección.

Figura 6

Macro F1 medio y desviación estándar muestral en cinco particiones



Nota. La escala vertical se limita a 0,930–0,945; las barras muestran una desviación estándar muestral. La diferencia entre modelos no se presenta como contraste de significación.

Tabla 19

Comparación de clasificadores en validación cruzada agrupada

Métrica	Regresión logística	Linear SVC
Accuracy media	0,9394	0,9390
Precisión macro media	0,9397	0,9405
Recall macro medio	0,9390	0,9381
Macro F1 medio	0,9392	0,9388
DE muestral de macro F1	0,0013	0,0020
Macro F1 OOF global	0,9392	0,9388
Iteraciones totales	105	239

Nota. Medias de cinco folds agrupados y desviación estándar muestral de macro F1; cifras del run robustness-1a94400d502b149f.

La comparación utiliza las mismas cinco asignaciones agrupadas y la misma configuración de representación para los dos candidatos. La Tabla 19 recoge las medias por *fold* y la desviación estándar muestral de macro F1. La regresión logística se seleccionó por la mayor media observada, sin tratar esa misma media como evaluación independiente posterior a la selección.

Macro F1 favorece una lectura que considera ambas clases. La clase `no_fake` y la clase `fake_news` proceden de transformaciones de fuentes externas con tamaños y estilos distintos; por ello, una métrica global no basta para explicar el comportamiento. La selección por macro F1 no elimina el posible sesgo de fuente, pero obliga a acompañar el resultado de precisión, recall, matriz y términos influyentes.

La Tabla 20 permite comprobar la variación entre particiones sin inferir significación estadística de diferencias pequeñas. El vocabulario TF-IDF y cada clasificador se ajustaron con el entrenamiento de su *fold*; la prueba correspondiente permaneció fuera de ese ajuste.

Tabla 20

Macro F1 y duración por partición agrupada

Modelo	Fold	n de prueba	Macro F1	Duración (s)
Regresión logística	1	8.881	0,9379	32,73
Regresión logística	2	8.880	0,9395	32,67
Regresión logística	3	8.885	0,9412	31,85
Regresión logística	4	8.881	0,9391	31,80
Regresión logística	5	8.881	0,9384	30,33
Linear SVC	1	8.881	0,9416	30,53
Linear SVC	2	8.880	0,9367	30,28
Linear SVC	3	8.885	0,9396	30,45
Linear SVC	4	8.881	0,9388	30,26
Linear SVC	5	8.881	0,9372	30,14

Nota. No hubo avisos de convergencia. Regresión logística usó lbfgs; Linear SVC usó liblinear con dual=True.

La desviación estándar muestral expresa la variación observada entre cinco particiones; no es una probabilidad de generalización universal. El macro F1 calcula el F1 de cada clase y promedia ambos sin ponderar por soporte. Ese resumen evita que el tamaño de una clase o fuente dominante oculte por completo la otra, pero un promedio global alto puede coexistir con una marcada diferencia por procedencia. Por eso se conservan las predicciones fuera de entrenamiento y el desglose archive2–LIAR junto a la media de los *folds*. La comparación de candidatos sobre estos mismos *folds* sigue siendo interna, no una evaluación independiente posterior a la selección.

5.2 Métricas del modelo seleccionado

Las métricas globales fuera de entrenamiento y el desglose por fuente se resumen en la Tabla 21. La media macro F1 de los *folds* es la cifra de selección; el resultado global se calcula sobre el conjunto completo de predicciones fuera de entrenamiento y constituye una síntesis descriptiva del mismo protocolo.

Tabla 21

Métricas fuera de entrenamiento de regresión logística por fuente

Métrica	Global OOF	archive2	LIAR
Registros	44.408	38.826	5.582
Accuracy	0,9394	0,9846	0,6245
Precisión macro	0,9396	0,9855	0,5951
Recall macro	0,9390	0,9837	0,5946
Macro F1	0,9392	0,9845	0,5949

Nota. Predicciones fuera de entrenamiento del modelo seleccionado; la cifra global no sustituye la media por fold utilizada para la selección.

La Tabla 21 distingue el rendimiento agregado fuera de entrenamiento del desglose por fuente. La precisión expresa qué proporción de predicciones positivas coincide con la etiqueta disponible; el recall, qué proporción de registros etiquetados como `fake_news` se recupera. Estas medidas describen el corpus derivado y los cinco *folds* agrupados, no una verificación factual ni una estimación para dominios ajenos.

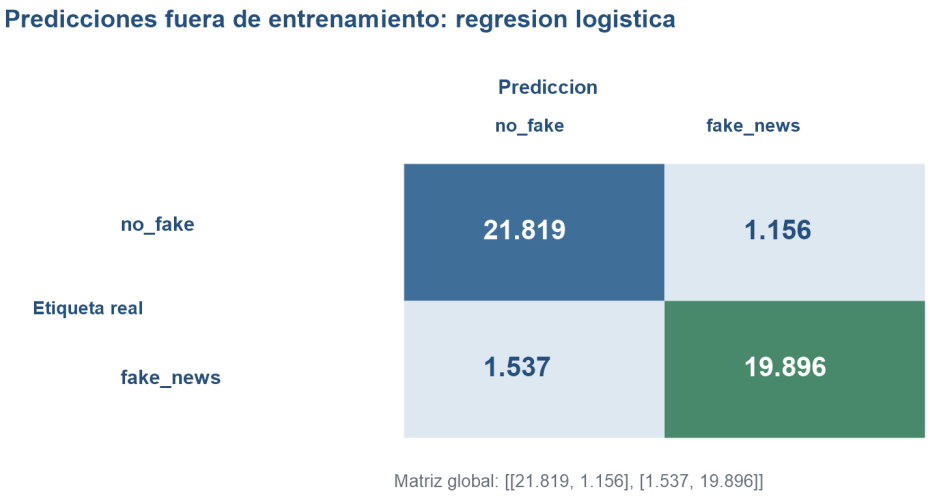
La interpretación debe considerar el tipo de salida. En Regresión Logística, el umbral se aplica a una probabilidad estimada; en Linear SVC, a una puntuación de decisión. Ambas magnitudes permiten ordenar decisiones binarias en su contexto técnico, pero no justifican comparaciones como si fuesen niveles de confianza calibrados. Por ello, el documento conserva los valores configurados y sus límites, sin atribuirles una propiedad universal de fiabilidad.

5.3 Matriz de confusión

La Figura 7 y la Tabla 22 muestran la matriz agregada de las 44.408 predicciones fuera de entrenamiento de regresión logística, el modelo seleccionado por macro F1 medio. La diagonal recoge aciertos frente a las etiquetas del conjunto de datos; las celdas externas recogen las discrepancias con esas mismas etiquetas, no verificaciones factuales.

Figura 7

Matriz de confusión global de regresión logística en predicciones OOF



Nota. Matriz agregada de 44.408 predicciones fuera de entrenamiento de la regresión logística, con etiquetas heredadas del corpus derivado.

Tabla 22

Matriz de confusión global fuera de entrenamiento

Etiqueta real / predicha	no_fake	fake_news
no_fake	21.819	1.156
fake_news	1.537	19.896

Nota. Filas: etiqueta heredada; columnas: predicción de la regresión logística; total: 44.408.

La lectura de esta matriz debe vincularse con dos decisiones de diseño conservadoras. El umbral 0,60 se aplica a la probabilidad estimada por Regresión Logística; el umbral 0,30 se aplica al valor de *decision_function* de Linear SVC y no representa una probabilidad. Ambos se fijaron manualmente en la configuración, no fueron aprendidos automáticamente y orientan la decisión a reducir falsos positivos en este experimento. No son universales, no se validaron en otros dominios y no constituyen por sí solos una garantía de fiabilidad.

Cada celda de la matriz resume predicciones fuera de entrenamiento del nuevo corpus. Un falso positivo es un registro etiquetado como no_fake que el modelo predice como fake_news; un falso negativo invierte esa discrepancia. Los CSV persistidos identifican source_id,

procedencia, etiqueta y puntuación, y permiten auditar cada fila sin atribuirle un canal o fecha de Telegram.

5.4 Análisis cualitativo de errores

Las predicciones fuera de entrenamiento permiten cuantificar dónde se concentran los errores. La Tabla 23 presenta falsos positivos y falsos negativos globales y por fuente, calculados a partir de las matrices persistidas. Estas cantidades no atribuyen por sí mismas una causa semántica a cada error.

Tabla 23

Errores fuera de entrenamiento por fuente

Procedencia	Falsos positivos	Falsos negativos
Global OOF	1.156	1.537
archive2	95	502
LIAR	1.061	1.035

Nota. Falsos positivos y negativos calculados sobre las matrices por fuente del mismo run.

La inspección responsable de un error comienza en las predicciones fuera de entrenamiento, localiza el texto original mediante `source_id` y compara etiqueta, puntuación, umbral y representación normalizada. Un ejemplo ilustrativo no reemplaza las métricas agregadas; ninguna de estas evidencias valida etiquetas externas o extrapolación a otro dominio.

En la matriz actual, los falsos positivos son registros con etiqueta heredada `no_fake` clasificados como `fake_news` por la regresión logística; los falsos negativos invierten la relación. Los recuentos deben relacionarse con el soporte de cada fuente y el umbral fijo. Un mayor número de errores en LIAR puede coexistir con declaraciones breves, otro vocabulario y una escala de origen binarizada. La memoria permite localizar esos casos, pero no contiene una intervención que mida cuánto aporta cada factor. Los ejemplos y términos del *baseline* histórico permanecen en un bloque separado para no atribuirlos al modelo principal.

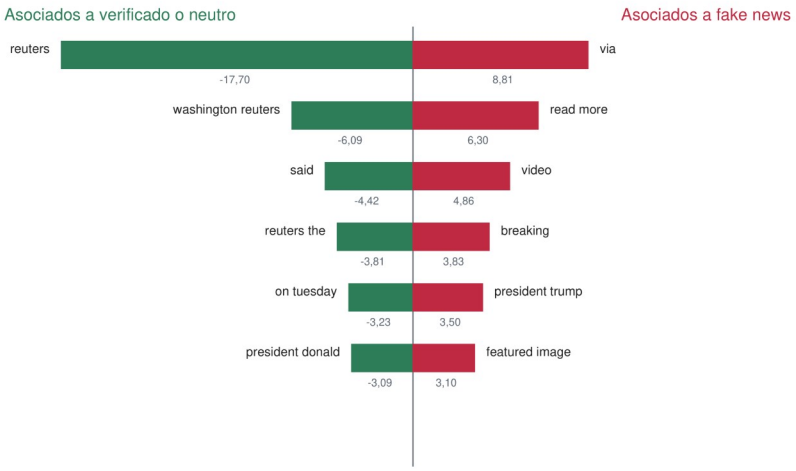
La lectura cualitativa del *baseline* anterior describía verdaderos positivos con marcas virales o referencias a vídeo, verdaderos negativos con estilo de agencia y menciones a Reuters, falsos positivos en algunas declaraciones breves de LIAR con léxico intenso y falsos negativos con apariencia factual o marcas de agencia. Son patrones ilustrativos del modelo y partición históricos, no cuadrantes recalculados para el experimento agrupado. Su interés metodológico reside en orientar preguntas sobre fuente y estilo editorial; no corrigen etiquetas externas ni autorizan extrapolar el catálogo de términos a nuevos dominios.

5.5 Interpretabilidad del modelo lineal

La Figura 8 y el Apéndice A contienen términos asociados a Linear SVC en el *baseline* histórico. No se presentan como explicación del modelo seleccionado en el run nuevo, ya que los coeficientes por *fold* no se agregaron en un estimador único. Los coeficientes históricos describen asociaciones dentro de su representación y corpus, no causas ni reglas de verificación.

Figura 8

Términos influyentes del baseline histórico de Linear SVC



Nota. Términos del artefacto linear_model_terms.json del baseline histórico; no son coeficientes agregados de la validación cruzada actual.

La Figura 8 muestra asociaciones léxicas del *baseline* histórico de Linear SVC. La presencia de marcas como Reuters sugiere dependencia de estilo editorial en esa versión, pero no explica por sí sola los resultados del experimento agrupado ni demuestra una causa de la etiqueta. La interpretación del modelo nuevo requeriría un análisis de coeficientes por partición o un estimador final definido para ese fin.

Los pesos se interpretan como señales del modelo lineal dentro del espacio TF-IDF configurado. Un peso positivo contribuye a desplazar la decisión hacia `fake_news` y uno negativo hacia `no_fake`, pero ningún término identifica una causa causal ni una regla suficiente por sí solo. El Apéndice A conserva una selección ampliada para verificar la Figura 8 frente al artefacto, manteniendo la advertencia sobre marcas editoriales.

Los coeficientes y ejemplos de error del *baseline* histórico permiten plantear preguntas de auditoría, pero no proporcionan una explicación causal de cada predicción. El signo y la magnitud de un peso dependen del vocabulario TF-IDF y del entrenamiento; términos como marcas editoriales pueden señalar procedencia. El catálogo se conserva en la Tabla A5 y no se atribuye al modelo seleccionado en el run agrupado.

5.6 Evaluación de transferencia entre fuentes

La transferencia se evaluó en dos sentidos, ajustando cada modelo únicamente con una fuente y probándolo en la otra. No se detectaron grupos con procedencia combinada, por lo que ninguna fila del corpus derivado se excluyó por ese criterio. Los resultados se limitan a estas dos fuentes y no representan una evaluación externa general.

La Tabla 24 recoge las cuatro evaluaciones de transferencia registradas.

Tabla 24

Macro F1 en transferencias entre archive2 y LIAR

Dirección	Modelo	Entrenamiento (n)	Prueba (n)	Macro F1
archive2 → LIAR	Regresión logística	38.826	5.582	0,5069
archive2 → LIAR	Linear SVC	38.826	5.582	0,5064
LIAR → archive2	Regresión logística	5.582	38.826	0,4741
LIAR → archive2	Linear SVC	5.582	38.826	0,5203

Nota. Se emplearon umbrales fijos; los grupos mixtos se excluyen cuando existen. Las métricas no validan transferencia a otros dominios.

Las cuatro matrices de transferencia y las predicciones por dirección están persistidas en los artefactos del run. La diferencia entre modelos en una dirección no debe confundirse con transferencia demostrada a Telegram u otro dominio no evaluado.

Las dos direcciones de transferencia no son intercambiables. Entrenar con noticias de archive2 y probar con declaraciones LIAR cambia el tamaño del entrenamiento y la forma del texto respecto a invertir origen y destino; también cambian la composición y la construcción de etiquetas. Los cuatro macro F1 proceden de ajustes específicos para cada dirección y candidato, con matrices y predicciones conservadas en el run. Su contraste con la validación cruzada mezclada observa simultáneamente varias diferencias de protocolo: no estima por sí solo el efecto de la deduplicación ni acredita funcionamiento sobre Telegram.

5.7 Validación automatizada

La suite actual de pytest registra 28 pruebas superadas y 0 fallidas en el informe JUnit de la ejecución del 24 de septiembre de 2026. Se incluyen doce pruebas del protocolo de robustez. El resultado histórico de 16 pruebas descrito en la versión previa queda únicamente como registro de esa versión y no se confunde con esta ejecución.

Las pruebas de Telegram usan `_FakeClient`; no existe una prueba específica de deduplicación persistente. El informe verifica software y contratos sintéticos, pero no demuestra validez estadística, generalización, rendimiento real en Telegram ni acceso a canales reales.

La evidencia automatizada se acompaña de comprobaciones de integridad del experimento: 44.408 predicciones por modelo, 44.408 filas asignadas, ningún grupo de duplicados repartido entre *folds* y ninguna advertencia de convergencia en los diez ajustes de validación cruzada. Las salidas del manifiesto conservan las iteraciones, duración y versiones. Estas verificaciones de ejecución no sustituyen la interpretación de amenazas a la validez.

5.8 Amenazas a la validez

La Tabla 25 reúne las amenazas a la validez de la comparación agrupada y de las dos pruebas de transferencia.

Tabla 25

Amenazas a la validez del estudio

Amenaza	Descripción observada	Límite o control
Efecto de fuente	Macro F1 global por fuente: archive2 0,9845; LIAR 0,5949.	Desglose OOF por fuente; no atribuir el resultado global solo a semántica.
Duplicación residual	La búsqueda MinHash es aproximada y puede omitir paráfrasis.	Agrupación de candidatos confirmados; límites declarados.
Validez de constructo	`no_fake` y `fake_news` heredan etiquetas y simplifican LIAR.	Se excluyen 7.184 categorías intermedias; no se afirma verificación independiente.
Validez estadística	Cinco folds estiman variación dentro del corpus disponible.	La media es comparación interna, no evaluación independiente posterior.
Validez externa	La transferencia se midió solo entre archive2 y LIAR.	No demuestra adaptación a otros dominios, épocas o Telegram.
Validez de implementación	Los cambios de código o datos pueden cambiar las salidas.	Hashes, manifiesto, asignaciones, modelos y pruebas persistidas.

Nota. Los controles reducen riesgos identificados, pero no eliminan el sesgo de fuente ni acreditan validez externa.

La agrupación reduce el solapamiento conocido, aunque la búsqueda aproximada de candidatos no garantiza detectar todas las paráfrasis. Las fuentes difieren en longitud, estilo y construcción de etiquetas; por ello, ni las métricas globales ni los términos históricos permiten aislar un efecto causal único de la deduplicación.

La amenaza de constructo aparece antes del modelado: las fuentes aportan etiquetas establecidas con criterios diferentes y la binarización de LIAR elimina granularidad. El riesgo de fuga se aborda con representantes exactos y *folds* por grupo confirmado, pero la búsqueda aproximada puede omitir relaciones. La amenaza de validez externa persiste aunque los contratos de software superen 28 pruebas, porque solo se observaron archive2 y LIAR. Por último, seleccionar el mejor candidato sobre los mismos *folds* significa que la media informa de una comparación interna. Una estimación independiente tras esa selección requeriría datos adicionales que no se han usado aquí.

5.9 Interpretación y límites de validez

La validez interna de los resultados principales se circunscribe al corpus derivado y a sus cinco *folds* agrupados. La comparación selecciona regresión logística por su mayor media macro F1, pero esa media se obtuvo en el mismo proceso de comparación y no constituye una estimación independiente posterior a seleccionar un ganador.

El desglose por fuente muestra una diferencia marcada entre archive2 y LIAR, consistente con distintas longitudes, formatos y etiquetas. La agrupación controla duplicados detectados, pero LSH no garantiza recuperar paráfrasis que no compartan banda. Cinco particiones permiten describir variabilidad interna; no resuelven la dependencia de dos fuentes ni justifican generalización universal.

Los mensajes de Telegram no participaron en el corpus, el entrenamiento ni la evaluación. Las pruebas entre archive2 y LIAR ofrecen resultados limitados a esas fuentes; antes de cualquier uso externo se requeriría un conjunto objetivo etiquetado, una anotación controlada y una evaluación específica de dominio.

La utilidad del nuevo experimento consiste en ofrecer una referencia reproducible y auditable con grupos controlados y predicciones fuera de entrenamiento. No constituye un detector universal, una validación de otros dominios ni un sistema desplegable.

Las amenazas identificadas se relacionan con decisiones observables. La procedencia se conserva en `dataset_name` y `source_id`; la binarización, en `original_label` y metadata; el estilo, en términos influyentes y ejemplos; y la dependencia de la partición, en configuración y manifiesto. Esta trazabilidad permite formular límites específicos y señalar qué evidencia sería necesaria para ampliar el alcance experimental.

5.10 Criterios de aceptación del protocolo

El protocolo de robustez se considera una referencia reproducible asociada al run robustness-1a94400d502b149f, a la fuente y al corpus derivado identificados por SHA-256, a la configuración y a las asignaciones de grupo documentadas. El criterio de selección es la media de macro F1 en cinco *folds* compartidos y no implica validez fuera de la evaluación interna.

La huella SHA-256 del corpus derivado es 184c707840a4bcff72dbef5e75e179152b630395c5f63f072db969bc1e79ea4e; la de la fuente

histórica de entrada es cfaf1b39c0260008e9328950649c9b83de2a9eadb478f631fb9cbc68d1b0a6b1. Cualquier cambio de fuente, reglas de duplicación, preprocesamiento, particiones, umbrales o dominio produciría una evaluación distinta y requeriría documentar un nuevo experimento.

El *baseline* histórico de una única partición permanece en un bloque separado. En esa versión se informó literalmente: «Linear SVC obtuvo el mejor resultado de los dos modelos comparados en la partición utilizada, con un macro F1 de 0,9424.» Ese resultado corresponde al run baseline-20260411T145313Z-a9ea359a; una auditoría posterior identificó solapamiento de textos normalizados entre entrenamiento y *holdout*. Como los protocolos no son equivalentes, no se atribuye causalmente la diferencia al control de duplicados. La configuración histórica se conserva en la Tabla A1, la comparación de modelos en la Tabla A2, las métricas por clase en la Tabla A3 y la matriz de confusión en la Tabla A4.

Una eventual aplicación a Telegram requeriría un corpus anotado, selección documentada, control de calidad y evaluación separada. Los resultados de transferencia entre archive2 y LIAR no se extrapolan a Telegram ni a otras fuentes.

6. Conclusiones y líneas futuras

El presente Trabajo Fin de Grado ha alcanzado el objetivo general de diseñar, implementar y evaluar un proceso experimental reproducible para la clasificación automática de *fake news* a partir de contenido textual. El resultado comprende la construcción y validación de un corpus derivado, el preprocesamiento de los textos, la representación mediante TF-IDF, la comparación de modelos supervisados en particiones agrupadas, la evaluación y la persistencia de los artefactos generados.

La principal aportación del trabajo es la construcción de una línea base reproducible, interpretable y auditable. Para ello se ha creado un corpus binario derivado mediante criterios explícitos de inclusión, exclusión, transformación de etiquetas y agrupación de duplicados. Asimismo, se ha implementado un flujo que conserva la procedencia de los registros, valida su estructura e integridad y permite reconstruir las condiciones de cada ejecución. La persistencia de configuraciones, métricas, predicciones, modelos, matrices de confusión y asignaciones de partición facilita la comparación entre experimentos y evita que los resultados queden reducidos a cifras aisladas difíciles de verificar.

La comparación entre regresión logística y Linear SVC muestra que ambos modelos lineales ofrecen un rendimiento elevado con representaciones TF-IDF dentro del protocolo agrupado. La regresión logística obtuvo el mayor macro F1 medio de cinco particiones, 0,9392 ± 0,0013, frente a 0,9388 ± 0,0020 de Linear SVC. Se trata de una comparación interna sobre las mismas particiones, no de una estimación independiente posterior a elegir el modelo. El macro F1 de 0,9424 de Linear SVC pertenece al *baseline* anterior y se conserva como referencia histórica, sin atribuir la diferencia entre protocolos únicamente a la deduplicación.

El análisis realizado evidencia que la evaluación de un clasificador de *fake news* no debe centrarse únicamente en la métrica final. La combinación de conjuntos de datos con distintas longitudes, formatos, estilos editoriales y procesos de anotación puede introducir señales de procedencia asociadas a las clases. En consecuencia, parte del rendimiento podría explicarse por el reconocimiento de patrones propios de las fuentes y no exclusivamente por características relacionadas con la veracidad del contenido. La inspección de los términos asociados a cada clase y de los ejemplos de error resulta, por tanto, necesaria para interpretar correctamente los resultados.

La formulación binaria ha permitido construir una referencia clara y manejable, pero ha requerido excluir categorías intermedias de LIAR cuya asignación a una de las dos clases habría sido arbitraria. Esta decisión reduce la granularidad del problema original. La validación cruzada en cinco particiones agrupadas limita el solapamiento de los grupos detectados entre entrenamiento y prueba, aunque la búsqueda aproximada de casi duplicados no garantiza identificar todas las paráfrasis. La transferencia entre archive2 y LIAR ofrece una comprobación adicional de dependencia respecto a la fuente; sus resultados no validan dominios externos.

El módulo de adquisición de mensajes de Telegram implementa la extracción y deduplicación bajo demanda de contenidos de canales públicos configurados. Sus pruebas automatizadas utilizan un cliente simulado y no acreditan acceso real a canales ni eficacia de clasificación en ese dominio. Los mensajes de Telegram carecen de etiquetas de referencia y no se han utilizado para entrenar ni evaluar los clasificadores. Esta separación constituye una decisión metodológica relevante: disponer de datos de una plataforma no implica que puedan incorporarse directamente a un experimento supervisado.

Entre las principales limitaciones se encuentran la dependencia de conjuntos de datos externos y de sus criterios de etiquetado, las señales de fuente y estilo editorial, la simplificación derivada

de la binarización y la cobertura aproximada de los casi duplicados. Aunque se evaluó transferencia en ambos sentidos entre archive2 y LIAR, el rendimiento observado fue inferior al de la validación interna y no se dispone de un corpus anotado de Telegram ni de una evaluación en otros dominios. Los resultados deben interpretarse dentro del corpus y de los protocolos documentados, sin atribuirles alcance universal.

Como líneas de trabajo futuro, se propone construir un corpus específico de Telegram mediante un protocolo documentado de selección, anotación y control de calidad; estudiar particiones temporales y fuentes adicionales; analizar los errores y la estabilidad de los términos influyentes; y estudiar la calibración de las salidas. Una vez consolidada la evaluación en el dominio objetivo, también podría abordarse la comparación con modelos de lenguaje preentrenados, manteniendo particiones y criterios equivalentes para garantizar un contraste metodológicamente válido.

En conclusión, el trabajo muestra que la utilidad de un sistema de clasificación automática no depende únicamente de alcanzar una métrica elevada, sino de poder explicar con qué datos, transformaciones, decisiones y limitaciones se ha obtenido. La trazabilidad y la interpretación crítica de los resultados constituyen, por tanto, aportaciones esenciales para valorar de forma rigurosa el alcance real del modelo desarrollado.

Bibliografía

Ahmed, H., Traore, I., & Saad, S. (2018). Detecting opinion spams and fake news using text classification. *Security and Privacy, 1*(1), e9. https://doi.org/10.1002/spy2.9

Bender, E. M., & Friedman, B. (2018). Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics, 6*, 587–604. https://doi.org/10.1162/tacl_a_00041

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*, 273–297. https://doi.org/10.1007/BF00994018

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

emineyetm. (2022). *Fake news detection datasets* [Data set]. Kaggle. https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM, 64*(12), 86–92. https://doi.org/10.1145/3458723

Gravanis, G., Vakali, A., Diamantaras, K., & Karadais, P. (2019). Behind the cues: A benchmarking study for fake news detection. *Expert Systems with Applications, 128*, 201–213. https://doi.org/10.1016/j.eswa.2019.03.036

Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. In C. Nédellec & C. Rouveirol (Eds.), *Machine learning: ECML-98* (pp. 137–142). Springer. https://doi.org/10.1007/BFb0026683

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns, 4*(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), Article 15. https://doi.org/10.1145/2382577.2382579

Lazer, D. M. J., Baum, M. A., Benkler, Y., Berinsky, A. J., Greenhill, K. M., Menczer, F., Metzger, M. J., Nyhan, B., Pennycook, G., Rothschild, D., Schudson, M., Sloman, S. A., Sunstein, C. R., Thorson, E. A., Watts, D. J., & Zittrain, J. L. (2018). The science of fake news. *Science, 359*(6380), 1094–1096. https://doi.org/10.1126/science.aao2998

Oshikawa, R., Qian, J., & Wang, W. Y. (2020). A survey on natural language processing for fake news detection. In *Proceedings of the 12th Language Resources and Evaluation Conference* (pp. 6086–6093). European Language Resources Association. https://aclanthology.org/2020.lrec-1.747/

Ozbay, F. A., & Alatas, B. (2020). Fake news detection within online social media using supervised artificial intelligence algorithms. *Physica A: Statistical Mechanics and its Applications, 540*, 123174. https://doi.org/10.1016/j.physa.2019.123174

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Pérez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018). Automatic detection of fake news. In *Proceedings of the 27th International Conference on Computational Linguistics* (pp. 3391–3401). Association for Computational Linguistics. https://aclanthology.org/C18-1287/

Potthast, M., Kiesel, J., Reinartz, K., Bevendorff, J., & Stein, B. (2018). A stylometric inquiry into hyperpartisan and fake news. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 231–240). Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1022

Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., & Dormann, C. F. (2017). Cross-validation strategies for data with

temporal, spatial, hierarchical, or phylogenetic structure. *Ecography, 40*(8), 913–929. https://doi.org/10.1111/ecog.02881

Ruchansky, N., Seo, S., & Liu, Y. (2017). CSI: A hybrid deep model for fake news detection. In *Proceedings of the 2017 ACM Conference on Information and Knowledge Management* (pp. 797–806). Association for Computing Machinery. https://doi.org/10.1145/3132847.3132877

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys, 34*(1), 1–47. https://doi.org/10.1145/505282.505283

Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake news detection on social media: A data mining perspective. *ACM SIGKDD Explorations Newsletter, 19*(1), 22–36. https://doi.org/10.1145/3137597.3137600

Silva, A., Luo, L., Karunasekera, S., & Leckie, C. (2021). Embracing domain differences in fake news: Cross-domain fake news detection using multi-modal data. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*(1), 557–565. https://doi.org/10.1609/aaai.v35i1.16134

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Tandoc, E. C., Lim, Z. W., & Ling, R. (2018). Defining fake news: A typology of scholarly definitions. *Digital Journalism, 6*(2), 137–153. https://doi.org/10.1080/21670811.2017.1360143

Thorne, J., & Vlachos, A. (2018). Automated fact checking: Task formulations, methods and future directions. In *Proceedings of the 27th International Conference on Computational Linguistics* (pp. 3346–3359). Association for Computational Linguistics. https://aclanthology.org/C18-1283/

Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science, 359*(6380), 1146–1151. https://doi.org/10.1126/science.aap9559

Wang, W. Y. (2017). “Liar, liar pants on fire”: A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 422–426). Association for Computational Linguistics. https://doi.org/10.18653/v1/P17-2067

Zhou, X., & Zafarani, R. (2020). A survey of fake news: Fundamental theories, detection methods, and opportunities. *ACM Computing Surveys, 53*(5), Article 109. https://doi.org/10.1145/3395046

Glosario

***Accuracy*.** Proporción de ejemplos correctamente clasificados sobre el total evaluado.

***Baseline*.** Modelo de referencia sencillo que permite comparar futuras mejoras con una base estable.

Corpus. Conjunto organizado de textos utilizado para entrenamiento o evaluación.

***Fake news*.** Contenido falso, engañoso o manipulado que se presenta con apariencia informativa.

***Holdout*.** Conjunto de validación separado del entrenamiento para estimar rendimiento.

LIAR. conjunto de datos de declaraciones políticas etiquetadas por niveles de veracidad.

Linear SVC. Clasificador lineal de vectores de soporte, adecuado para datos textuales de alta dimensión.

Macro F1. Promedio no ponderado del F1 calculado por clase.

N-grama. Secuencia de una o varias palabras consecutivas utilizada como característica textual.

Source bias. Sesgo producido cuando un modelo aprende marcas de procedencia de los datos en lugar de señales propias de la tarea.

TF-IDF. Esquema de ponderación que combina frecuencia del término e inversa de frecuencia documental.

Umbral de decisión. Valor que transforma una puntuación continua del modelo en una etiqueta discreta.

Apéndice A. Evidencia histórica del *baseline* de una partición

Este apéndice conserva únicamente la configuración y los resultados del run histórico baseline-20260411T145313Z-a9ea359a. Sus 12.626 ejemplos de *holdout* y su macro F1 de 0,9424 no forman parte de la validación cruzada agrupada actual ni se interpretan como un efecto aislado de la deduplicación.

Tabla A1

Configuración principal del baseline histórico

Parámetro	Valor
Dataset	\tfg_informatica_fake_news\data\labeled\fake_news_unified.csv.gz
Columna de texto	`normalized_text`
Columna de etiqueta	Label
Métrica de selección	`macro_f1`
Mejor modelo	`linear_svc`
Tipo de puntuación	`decision_function`
Umbral aplicado	0,30
Filas de entrenamiento	37.877
Filas de validación	12.626
Partición	test_size = 0,25; random_state = 42
Representación	TF-IDF con ngram_range = (1, 2)
Vocabulario	min_df = 1; max_features = 5000
Regresión Logística	C = 1,0; max_iter = 1000; class_weight = balanced
Linear SVC	C = 1,0; class_weight = balanced
Umbral probabilístico	0,60
Umbral de decisión	0,30
Directorios	reports/training_runs y models/training_runs

Nota. Parámetros de training_config.json y training_manifest.json; no describen el protocolo agrupado actual.

Tabla A2

Comparación de modelos del baseline histórico

Modelo	Accuracy	Precisión macro	Recall macro	Macro F1	Umbral
Regresión logística	0,9410	0,9403	0,9417	0,9408	0,60
Linear SVC	0,9426	0,9420	0,9439	0,9424	0,30

Nota. Holdout histórico de 12.626 ejemplos; cifras sin recalcular.

Tabla A3

Métricas históricas por clase

Métrica	Global	Clase 0 (hist.)	Clase 1 (hist.)
Accuracy	0,9426	—	—
Precisión	0,9420	0,9179	0,9660
Recall	0,9439	0,9625	0,9253
F1	0,9424	0,9397	0,9452
Soporte	12.626	5.868	6.758

Nota. Clase 0 (hist.) equivale a verificado_o_neutro y clase 1 a fake_news; la clase negativa actual se denomina no_fake.

Tabla A4

Matriz de confusión del baseline histórico

Real / predicho	Clase 0 (hist.)	Clase 1 (hist.)
Clase 0 (hist.)	5.648	220
Clase 1 (hist.)	505	6.253

Nota. Clase 0 (hist.) equivale a verificado_o_neutro y clase 1 a fake_news. Recuentos del holdout histórico, no del corpus derivado.

Tabla A5

Términos influyentes del modelo histórico Linear SVC

Clase asociada	Término	Peso	Lectura metodológica
Clase 0 (hist.)	reuters	-17,699864	Marca editorial especialmente distintiva.
Clase 0 (hist.)	washington reuters	-6,092424	Combinación de procedencia y localización.
Clase 0 (hist.)	said	-4,419298	Fórmula frecuente en textos de agencia.
Clase 0 (hist.)	reuters the	-3,812113	Patrón de estilo asociado a la fuente.
Clase 0 (hist.)	on tuesday	-3,229798	Construcción temporal del corpus.
Clase 0 (hist.)	president donald	-3,087900	Expresión contextual con correlación observada.
Clase 1 (hist.)	via	8,812112	Marcador de formato con asociación positiva.
Clase 1 (hist.)	read more	6,302467	Llamada editorial o de navegación.
Clase 1 (hist.)	video	4,863642	Término de formato asociado a registros positivos.
Clase 1 (hist.)	breaking	3,833543	Expresión de énfasis del corpus.
Clase 1 (hist.)	president trump	3,504171	Asociación dependiente de la composición del corpus.
Clase 1 (hist.)	featured image	3,098978	Marca de plantilla o formato editorial.

Nota. Clase 0 (hist.) equivale a verificado_o_neutro y clase 1 a fake_news. Pesos del baseline histórico, no coeficientes agregados del protocolo actual.